

Order Reinforcements

Contents

1. Analysis	3
1.1 The problem: identification and background	3
1.2.1 Description of the current system	3
1.2.2 Problems with current system	4
1.3 Objectives.....	4
1.4 Data modelling.....	6
2. Design.....	7
2.1 Hierarchy chart.....	7
2.2 Normalised entity relationship diagram	7
2.2.1 Entity description in standard notation	8
2.2.2 Design data dictionary	8
2.3. Form and algorithm design	10
2.3.1 Main form (products tab)	10
2.3.2 Cart form	19
2.3.3 Order Details form	22
2.3.4 Class diagram (methods list)	26
2.4 Report design	27
2.4.1 Sales chart	27
2.4.2 Order report.....	28
2.4.3 Receipt	30
3. Testing.....	32
3.1 Test plan.....	32
3.2 Test results.....	37
4. Evaluation	38
4.1 Objectives comparison.....	38
4.2 Improvements.....	38
4.3 Analysis of 3 rd party feedback	39
5. Technical solution	40
5.1 Class Main	40
5.2 Class Account	51
5.3 Class Addresses	55
5.4 Class Cart.....	60

5.5 Class Checkout	67
5.6 Class Edit Details	70
5.7 Class Manage Users	73
5.8 Class Order Details	77
5.9 Class Sign In	82
5.10 Class DBConnector	89
5.11 Class Program.....	90

1. Analysis

1.1 The problem: identification and background

Last two summers I've been working at Cumberland Reinforcements Ltd, they specialise in the supply and fabrication of concrete reinforcing for the construction industry. Orders have to be taken by a manager on call, which are transferred to paper and then taken to the main construction site. Customers can request the rebars (metal bars used to reinforce concrete) to be in desired diameter, length. They can choose how many of them they want.

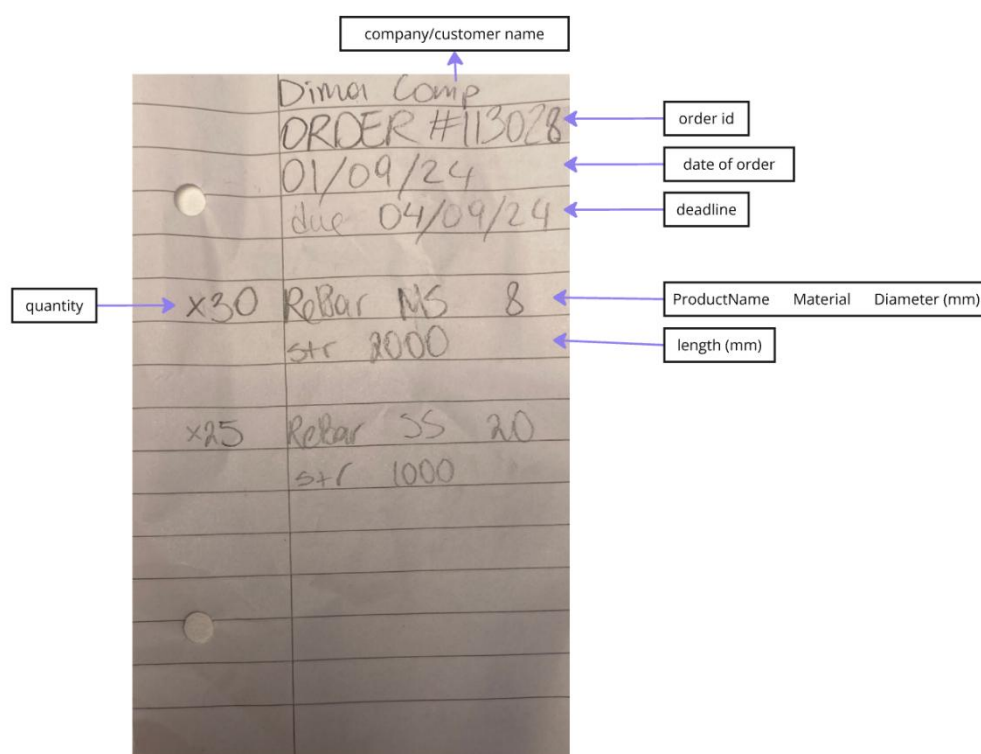
Currently, computers are only used to calculate earnings. In the past there have been issues such as multiple customers trying to call the manager at the same time, incorrectly transferred to paper dimensions or other details of the order, it could also be due to the manager hearing the order wrong. All this can be fixed with the implementation of software.

With the help of my manager, I will spend time to learn the current system to find any problems in the process and see what improvements can be made.

1.2.1 Description of the current system

Customers contact a manager on call via phone to place orders: During the call, customers provide what they would like to get and can ask for different variations of rebars, then the manager notes down the order details manually on paper. Total price per product is calculated by multiplying price per meter (depends on material and diameter), length, quantity and VAT value (1.20). The paper records are taken to the main construction site where the production team begins preparing rebars and assembling the order.

Annotated picture of how managers take orders:



1.2.2 Problems with current system

Customer interactions are entirely phone-based, leaving no digital record of the orders for reference or verification. Miscommunication over the phone may result in incorrect order details being noted, such as wrong dimensions, quantities. Managers and employees spend extra time managing phone calls, and paperwork which could be automated.

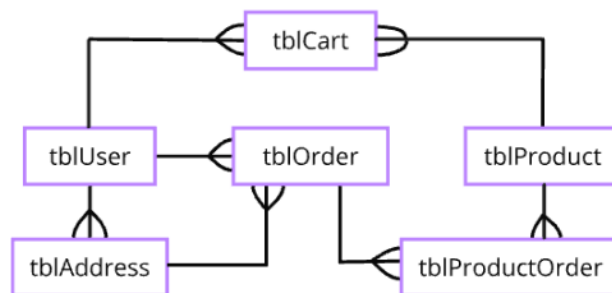
1.3 Objectives

1. A form to either log in, using e-mail and password, or register, using their first name, surname, e-mail, phone number, company name (optional) and desired password, which should be confirmed in a different field.
 - 1.1. A button to reveal password, while pressing on the button.
 - 1.2. Register controls should be hidden by default.
 - 1.3. An option to 'remember me' and 'forgot password'.
 - 1.4. Validation of fields, with a corresponding error message if not valid.
 - 1.5. Based on user being customer or manager, different versions of main form will be shown after logging in.
2. Main form for managers will include statistics and two tables: orders and users. Both customers and managers can order products.
 - 2.1. All functions stated previously should be located on different tab-pages.
 - 2.2. By double-clicking or pressing on an allocated button on one of the orders, details can be viewed on a different form.
 - 2.3. Managers can edit users' details and addresses by pressing on corresponding buttons, which take you to different forms.
 - 2.4. Check box for viewing either customers or managers in users' table.
 - 2.5. Order reports between specified dates and a chart showing total sales per month.
 - 2.6. Dynamically created buttons for each product. Pressing on one will show product's details.
 - 2.6.1. If product is out of stock, it will be stated and you would not be able to order it.
 - 2.6.2. You can select desired length (between 0.3 and 6.0 meters) and quantity of rebars. Depending on the amount in stock of a selected product and specified length, maximum quantity is stated.
 - 2.6.3. Total price is shown.
 - 2.6.4. If everything is valid, product can be added to cart by pressing on a button.
 - 2.7 Two buttons which can be accessed from each tab-page: account and cart.
3. Order details can be viewed on a different form (if user is manager).
 - 3.1. Order can be marked as complete/uncomplete by using a check box.
 - 3.2. Order details are shown: date of order, date of completion, delivery address.
 - 3.3. Customer details are shown: name, e-mail, phone number and company name, if applicable.
 - 3.4. List view with products and their details, such as quantity, diameter, material, length and total price per product.
 - 3.5. Subtotal, VAT and total are calculated and shown.
 - 3.6. Dynamically created receipts can be viewed.

4. Users can be added/deleted/updated on a separate form (by managers).
 - 4.1. Combo box to choose a user to manage.
 - 4.2. First name, surname, e-mail, phone number, company name of a chosen user can be chosen.
 - 4.3. Option to set user as a manager using check box.
 - 4.4. 4 buttons: clear (resets all fields), add, update, delete.
 - 4.5. For changes to take place all fields need to be valid, corresponding error message will be shown if not.
 - 4.6. If a new user is added, given option to associate an address.
5. A form where addresses can be added/deleted/updated (by managers).
 - 5.1. Combo box to choose a user, whose addresses to manage.
 - 5.2. One user can have multiple addresses.
 - 5.3. When a user is chosen, list view will display all addresses associated with that user.
 - 5.4. First line, town and postcode can be typed in.
 - 5.5. 4 buttons: clear (resets all fields), add, update, delete.
 - 5.6. For changes to take place all fields need to be valid, corresponding error message will be shown if not
6. A form responsible for personal account management.
 - 6.1. Name and status (customer/manager) of the user logged in are displayed.
 - 6.2. 3 buttons: edit, address, sign out.
 - 6.2.1. Edit button takes user to a new form: all details can be changed, including password. All fields are validated.
 - 6.2.2. Address button takes user to address form. Combo box is disabled, meaning user can only add/delete/update their own addresses.
 - 6.2.3. Sign out button asks for confirmation first, if committed, program restarts, displaying sign in form again.
 - 6.3. List view with every order done by the logged in user and.
 - 6.4. Double-clicking on one of them will take the user to order details form, order cannot be marked as complete/uncomplete if the form is entered from here.
7. A form for cart which will display its contents.
 - 7.1. If cart is empty, corresponding message will be shown.
 - 7.2. Otherwise, each product from the cart will be dynamically created with an image, its name, quantity, length and price.
 - 7.3. Quantity can be changed using numeric-up-down, maximum quantity is set so users cannot order more than there is in stock.
 - 7.4. Every product can be deleted by a dynamically created button next to it. User will be asked for confirmation.
 - 7.5. Totals (both excluding and including VAT) are calculated and shown.
 - 7.6. 'Checkout' button.

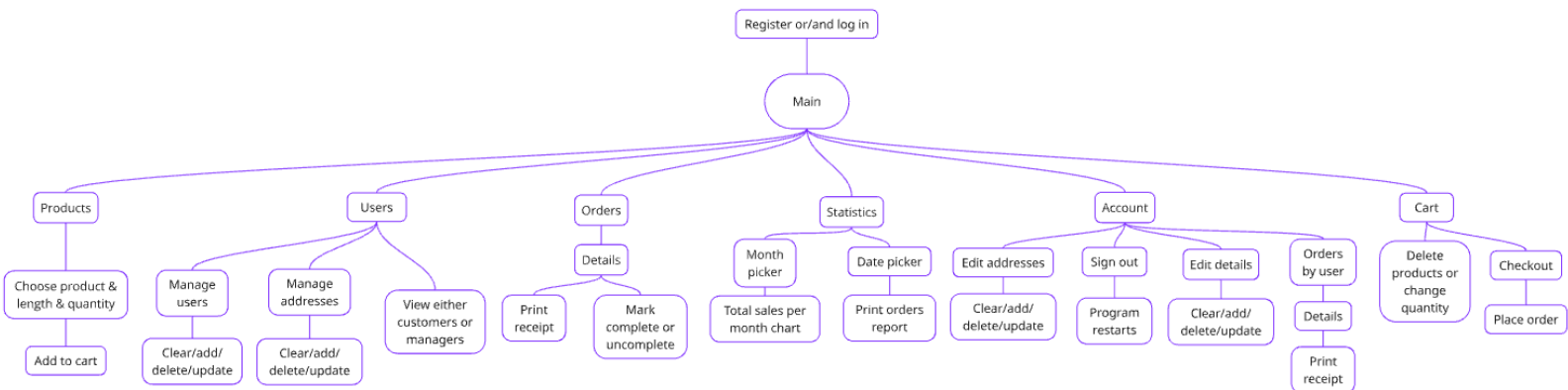
8. Checkout form with summary, delivery address, and payment details.
 - 8.1. Any delivery address can be chosen that is associated with a user.
 - 8.2. Card holder name, card number, expiration date and CVC text fields must be filled.
 - 8.3. Subtotal, VAT and total are shown.
 - 8.4. A button to place order checks if fields are valid, and if are, “money is taken” and order is placed. Cart is emptied after that. All fields on main form are updated.

1.4 Data modelling

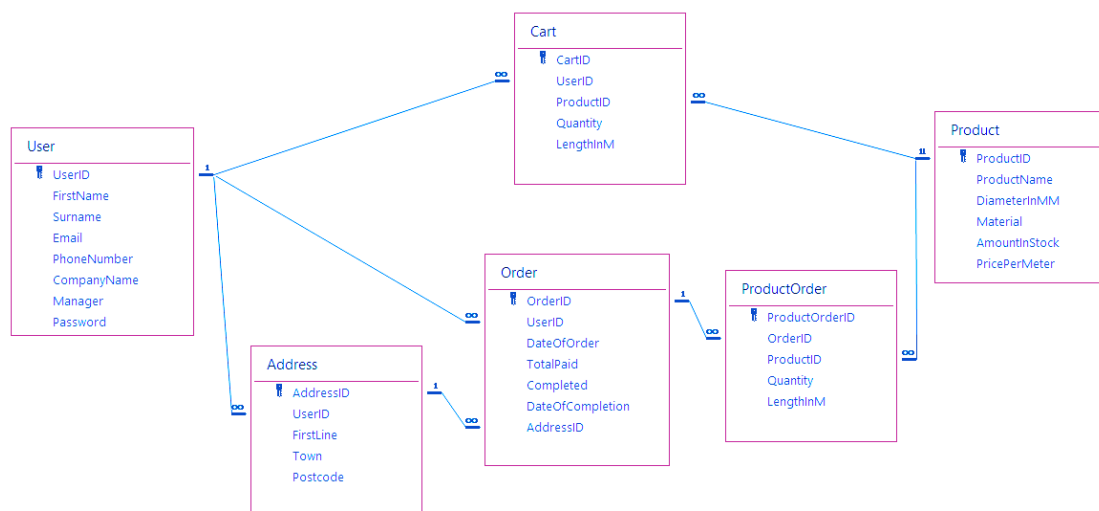


2. Design

2.1 Hierarchy chart



2.2 Normalised entity relationship diagram



2.2.1 Entity description in standard notation

- User (UserID, FirstName, Surname, Email, PhoneNumber, CompanyName, Manager, Password)
- Address (AddressID, UserID, FirstLine, Town, Postcode)
- Order (OrderID, UserID, DateOfOrder, TotalPaid, Completed, DateOfCompletion, AddressID)
- Product (ProductID, ProductName, DiameterInMM, Material, AmountInStock, PricePerMeter)
- ProductOrder (ProductOrderID, OrderID, ProductID, Quantity, LengthInM)
- Cart (CartID, UserID, ProductID, Quantity, LengthInM)

2.2.2 Design data dictionary

2.2.2.1 Table User

Field Name	Data Type	Description
UserID	int	primary key
FirstName	varchar(255)	stores user's first name
Surname	varchar(255)	stores user's surname
Email	varchar(255)	stores user's e-mail; used for logging in; in some cases used to help software identify what user is logged in
PhoneNumber	varchar(255)	user's phone number; i.e. 09999999999
CompanyName	varchar(255)	optional; user's company name
Manager	bool	if set to false = user is a customer; if set to true = user is a manager and additional functionality is enabled in software
Password	varchar(255)	user's hashed password; empty field ("") indicates user being created by manager

2.2.2.2 Table Address

Field Name	Data Type	Description
AddressID	int	primary key
UserID	int	foreign key from User table; associates address with a certain user; users can have multiple addresses for delivery purposes
FirstLine	varchar(255)	first line of address; i.e. 20a Aleksandra Road
Town	varchar(255)	town of address; i.e. Bournemouth
Postcode	varchar(255)	postcode of address; i.e. BH12 9FR

2.2.2.3 Table Order

Field Name	Data Type	Description
OrderID	int	primary key
UserID	int	foreign key from User table; indicates what user completed order
DateOfOrder	dateTime	stores date of order
TotalPaid	money	stores total money spent by user; includes VAT
Completed	bool	indicates if order is completed yet
DateOfCompletion	dateTime	if uncomplete = empty field (""); else date of completion
AddressID	int	foreign key from Address table; stores delivery address for order

2.2.2.4 Table Product

Field Name	Data Type	Description
ProductID	int	primary key
ProductName	varchar(255)	stores product's name
DiameterInMM	int	stores product's diameter in millimetres
Material	varchar(255)	stores product's material name
AmountInStock	int	stores amount of product left in stock; 1 bar = 12 meters; automatically decreased if ordered
PricePerMeter	money	stores product's cost per meter; excludes VAT

2.2.2.5 Table ProductOrder

Field Name	Data Type	Description
ProductOrderID	int	primary key
OrderID	int	foreign key from Order table; represents what order this ordered product is associated with
ProductID	int	foreign key from Product table; represents what product is ordered
Quantity	int	stores quantity of ordered product
LengthInM	int	stores length of ordered product in meters; default length = 1; often converted to millimetres by software for data manipulation

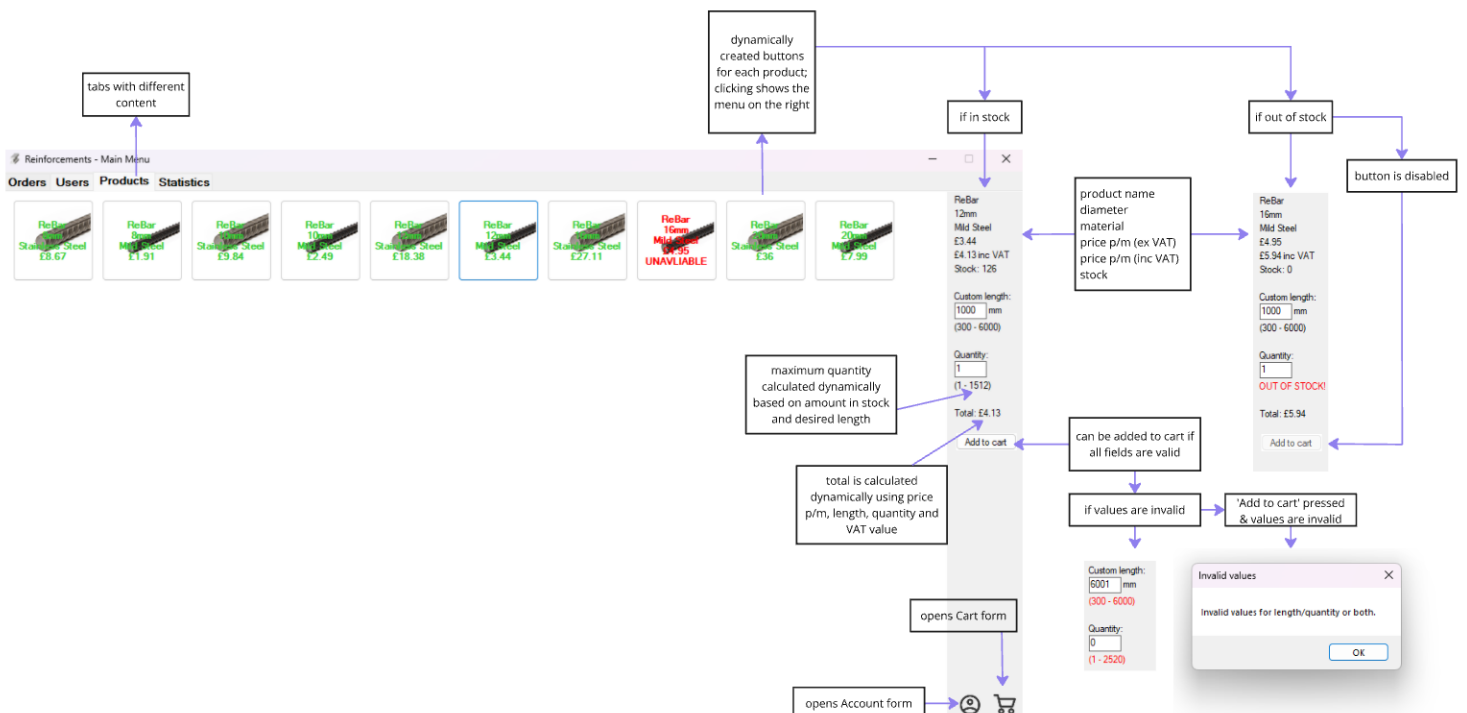
2.2.2.6 Table Cart

Field Name	Data Type	Description
CartID	int	primary key
UserID	int	foreign key from User table; represents whose cart includes product; when order is placed by user, records with corresponding UserID are deleted
ProductID	int	foreign key from Product table; indicates what product is in cart
Quantity	int	stores amount of product in cart
LengthInM	int	stores length of product in cart in meters

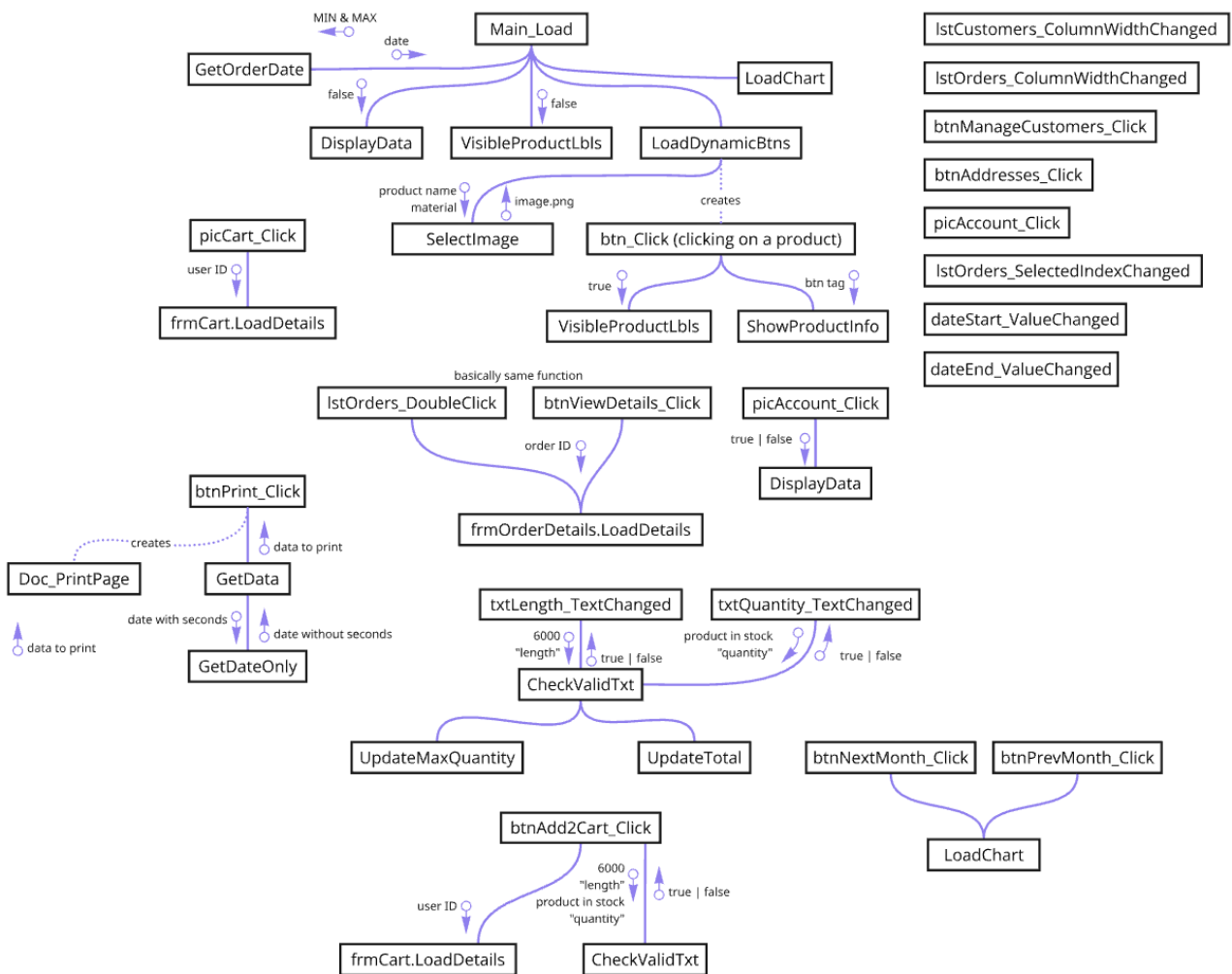
2.3. Form and algorithm design

2.3.1 Main form (products tab)

2.3.1.1 Form screen shot



2.3.1.2 Structure chart



2.3.1.3 Pseudo code

```
selectedMonth ← current date
FUNC Main_Load
    DisplayData(FALSE)
    IF (manager ← FALSE)
        remove tabs for managers only
    END IF
    VisibleProductLbls(FALSE)
    LoadDynamicBtns
    LoadChart
    dateStart.Value ← GetOrderDate("MIN")
    dateEnd.Value ← GetOrderDate("MAX")
END FUNC
```

```

FUNC LoadChart
    daysInMonth ← get how many days in selected month
    salesData ← new Dictionary<int, double>
    FOR i ← 1 TO daysInMonth
        salesData[i] ← 0
    END FOR
    create chart "Sales"
    dr ← "SELECT DAY(DateOfOrder), SUM(TotalPaid) FROM tblOrder WHERE
YEAR(DateOfOrder) = '" + selectedMonth.Year + "' AND MONTH(DateOfOrder) = '" +
selectedMonth.Month + "' GROUP BY DAY(DateOfOrder) ORDER BY DAY(DateOfOrder)"
    WHILE (dr.Read())
        orderDay ← DAY(DateOfOrder)
        totalPaid ← SUM(TotalPaid)
        salesData[orderDay] ← totalPaid
    END WHILE
    FOR EACH entry IN salesData
        add bar to chart where X = date and Y = totalPaid
    END FOR
END FUNC

FUNC VisibleProductLbls(bool)
    every label responsible for displaying data about product ← bool
END FUNC

FUNC LoadDynamicBtns
    dr ← "SELECT ProductID, ProductName, DiameterInMM, Material, PricePerMeter,
AmountInStock FROM tblProduct ORDER BY DiameterInMM"
    WHILE (dr.Read())
        create new button and add to Products tab
        btn.Tag ← ProductID
        btn.Text ← ProductName, DiameterInMM, Material, PricePerMeter
        btn.Image ← SelectImage(ProductName, Material)
    END WHILE
END FUNC

FUNC btn_Click()
    VisibleProductLbls(TRUE)
    ShowProductInfo(button tag)
END FUNC

FUNC ShowProductInfo(button tag)
    productID ← button tag
    dr ← "SELECT ProductName, DiameterInMM, Material, PricePerMeter, AmountInStock
FROM tblProduct WHERE ProductID = " + productID
    dr.Read()
    labels show text with product info
    price ← PricePerMeter
    priceVAT ← rounded to 2dp (PricePerMeter * 1.2)
    labels show text with product price
    productInStock ← AmountInStock
    IF (productInStock = 0)
        cannot order this product
    END IF
END FUNC

```

```
FUNC SelectImage(productName, material)
    IF (productName = "rebar")
        IF (material = "mild steel")
            return image in png containing rebar made out of mild steel
        ELSE IF (material = "stainless steel")
            return image in png containing rebar made out of stainless steel
        END IF
    otherwise return null
END FUNC

FUNC GetOrderDate(maxOrMin)
    dr ← "SELECT " + maxOrMin + "(DateOfOrder) FROM tblOrder"
    dr.Read()
    return DateOfOrder
END FUNC

FUNC GetBoolEmoji(boolean)
    IF (boolean = "true")
        RETURN "✓"
    END IF
    RETURN "✗"
END FUNC

FUNC DisplayData(v)
    TRY
        currentFile ← "temp.txt"
        dr ← "SELECT UserID, Manager FROM tblUser WHERE Email = '" +
currentFile.ReadLine() + "'"
        dr.Read()
        userID ← UserID
        IF (Manager = "True")
            manager ← TRUE
        END IF
        dr ← "SELECT OrderID, UserID, DateOfOrder, TotalPaid, Completed,
DateOfCompletion, AddressID FROM tblOrder ORDER BY Completed DESC"
        WHILE (dr.Read())
            complete Order list view with items from database
        END WHILE

        dr ← "SELECT UserID, FirstName, Surname, Email, PhoneNumber,
CompanyName FROM tblUser WHERE Manager = " + v + " ORDER BY UserID DESC"
        WHILE (dr.Read())
            complete Users list view with items from database, where users are
customers if v = false and managers if v = true
        END WHILE
    CATCH
        show error message
    END TRY
END FUNC
```

```
FUNC GetDateOnly(originalDBDate)
    IF (originalDBDate ≠ null)
        fullDate ← originalDBDate
        return fullDate in format dd/MM/yyyy
    END IF
    return "✕" if, for example, DateOfCompletion isnt set
END FUNC

FUNC lstCustomers_ColumnWidthChanged()
    IF (width of columns in Users list view > MaxColumnWidth)
        width of columns in Users list view ← MaxColumnWidth
    END IF
END FUNC

FUNC lstOrders_ColumnWidthChanged()
    IF (width of columns in Orders list view > MaxColumnWidth)
        width of columns in Orders list view ← MaxColumnWidth
    END IF
END FUNC

FUNC btnManageCustomers_Click()
    IF (NOT frmMUOpen)
        frmMUOpen ← TRUE
        open frmManageUsers
    ELSE
        bring form to front
    END IF
END FUNC

FUNC btnAddresses_Click(sender, e)
    IF (NOT frmAddrOpen)
        frmAddrOpen ← TRUE
        open frmAddresses
    ELSE
        bring form to front
    END IF
END FUNC

FUNC picAccount_Click(sender, e)
    IF (NOT frmAccOpen)
        frmAccOpen ← TRUE
        open frmAccount
    ELSE
        bring form to front
    END IF
END FUNC
```

```
FUNC checkBox1_CheckedChanged()
    IF (checkBox1.Checked)
        DisplayData(TRUE)
    ELSE
        DisplayData(FALSE)
    END IF
END FUNC

FUNC lstOrders_SelectedIndexChanged()
    set selectedOrderID depending on what item is selected
END FUNC

FUNC btnViewDetails_Click()
    IF (order isnt selected)
        show error message
    ELSE
        show frmOrderDetails
        frmOrderDetails.LoadDetails(selectedOrderID)
    END IF
END FUNC

FUNC lstOrders_DoubleClick()
    IF (order is selected)
        show frmOrderDetails
        frmOrderDetails.LoadDetails(selectedOrderID)
    END IF
END FUNC

FUNC picCart_Click(sender, e)
    IF (frmCartOpen = FALSE)
        frmCartOpen ← TRUE
        show frmCart
        frmCart.LoadDetails(userID)
    ELSE
        bring form to front
    END IF
END FUNC

FUNC btnPrint_Click()
    doc ← new PrintDocument
    header ← string with columns: Order ID, Customer, Date, Paid
    stringToPrint ← GetData()
    documentContents ← stringToPrint
    printPreviewDialog1.Document = doc
    show printPreviewDialog1
END FUNC
```

```

FUNC GetData()
    totalSales ← 0
    dataToPrint ← ""
    dr ← SELECT OrderID, FirstName, Surname, DateOfOrder, TotalPaid
        FROM tblOrder, tblUser
        WHERE tblOrder.UserID = tblUser.UserID
        AND tblOrder.DateOfOrder >= startDate AND tblOrder.DateOfOrder <= endDate
        ORDER BY DateOfOrder
    dataToPrint ← header
    WHILE (dr.Read())
        totalSales ← totalSales + dr[4]
    append formatted line to dataToPrint with OrderID, Customer Name,
    Date, and TotalPaid
    END WHILE
    IF (dataToPrint = header)
        append "NO DATA FOUND!" to dataToPrint
    ELSE
        append "Total Sales: £" + totalSales to dataToPrint
    END IF
    RETURN dataToPrint
END FUNC

FUNC Doc_PrintPage()
    draw title text "REINFORCEMENTS: ORDERS REPORT"
    draw date range text
    measure stringToPrint to get charactersOnPage and linesPerPage
    draw stringToPrint
    stringToPrint ← remaining part after charactersOnPage
    IF (stringToPrint is not empty)
        stringToPrint ← documentContents
    ELSE
        stringToPrint ← header + stringToPrint
    END IF
END FUNC

FUNC dateStart_ValueChanged()
    startDate ← value of dateStart
END FUNC

FUNC dateEnd_ValueChanged(sender, e)
    endDate ← value of dateEnd
END FUNC

FUNC CheckValidTxt(max, input)
    IF (input = "length")
        TRY
            IF (chosenLength ≥ 300 AND chosenLength ≤ max)
                UpdateMaxQuantity()
                UpdateTotal()
                RETURN TRUE
            END IF
        CATCH
            RETURN FALSE
        END TRY
    END IF
END FUNC

```



```

ELSE IF (input = "quantity")
    TRY
        MMInStock ← max * 12000
        maxQuantity ← MMInStock / chosenLength
        IF (quantity ≥ 1 AND quantity ≤ maxQuantity)
            UpdateTotal()
            RETURN TRUE
        END IF
    CATCH
        RETURN FALSE
    END TRY
END IF
RETURN FALSE
END FUNC

FUNC UpdateTotal()
    total ← round to 2dp(1.2 * (price * (chosenLength / 1000) * chosenQuantity))
    update total label
END FUNC

FUNC UpdateMaxQuantity()
    IF (productInStock ≠ 0)
        maxQuantity ← (productInStock * 12000) / chosenLength
        update max quantity label
    END IF
END FUNC

FUNC txtLength_TextChanged()
    IF (NOT CheckValidTxt(6000, "length"))
        colour of label is set to red
    ELSE
        black
    END IF
END FUNC

FUNC txtQuantity_TextChanged(sender, e)
    IF (NOT CheckValidTxt(productInStock, "quantity"))
        colour of label is set to red
    ELSE
        black
    END IF
END FUNC

FUNC btnAdd2Cart_Click()
    IF (CheckValidTxt(6000, "length") AND CheckValidTxt(productInStock, "quantity"))
        DoDML($"INSERT INTO tblCart (UserID, ProductID, Quantity, LengthInM)
VALUES ('{userID}', '{productID}', '{txtQuantity.Text}', '{Convert.ToDouble(txtLength.Text) /
1000}')")
        IF (frmCartOpen = TRUE)
            refresh form
        ENDIF
    ENDIF

```

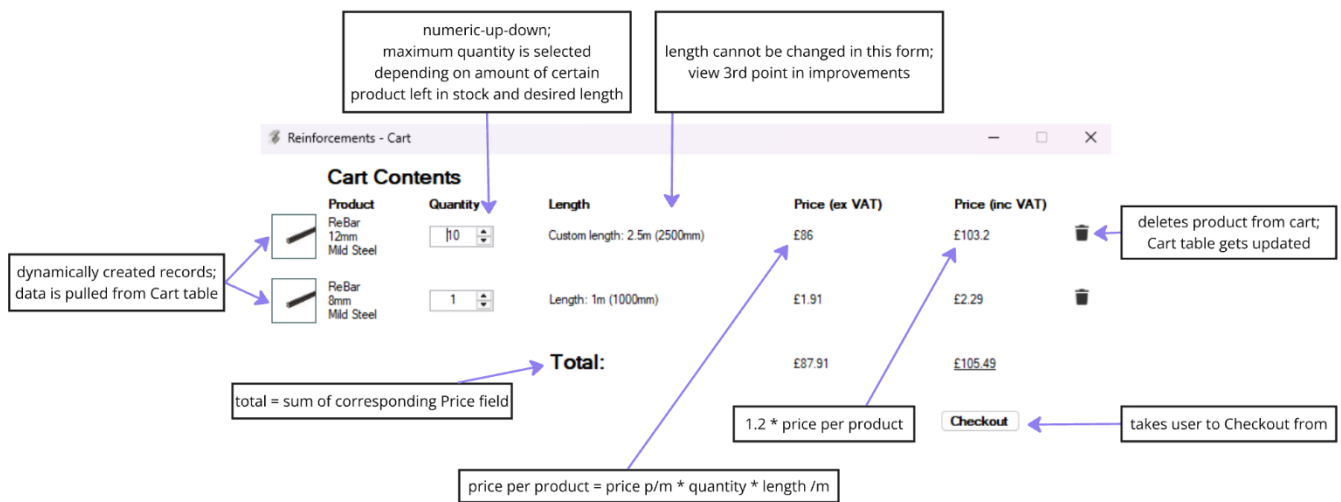
```
        ELSE
            error message is shown: "Invalid values for length/quantity or both."
        ENDIF
    END FUNC

FUNC btnNextMonth_Click(sender, e)
    increase chosen month by one month
    LoadChart()
FUNC

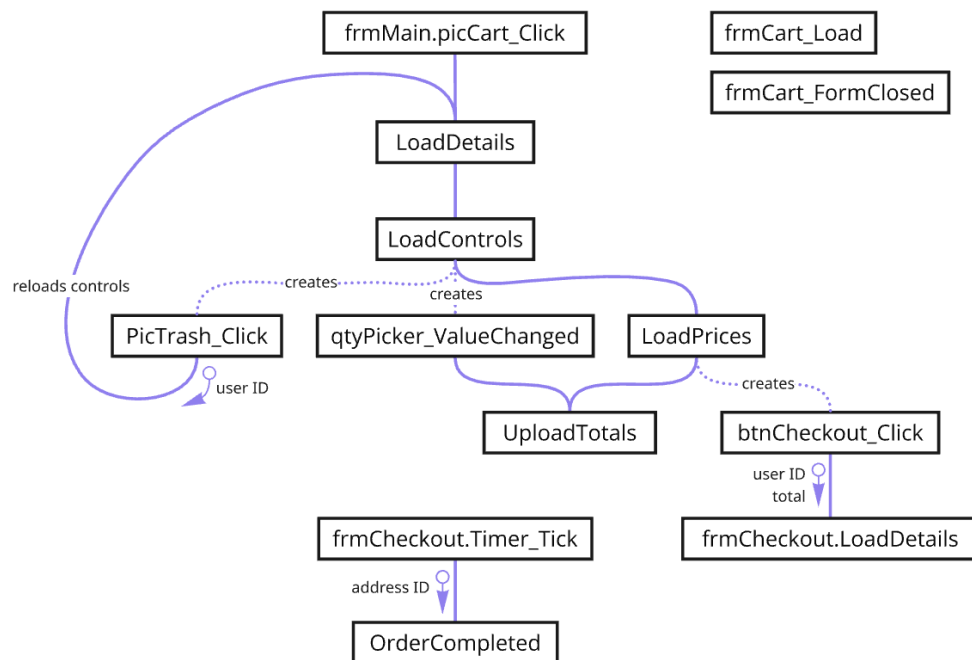
FUNC btnPrevMonth_Click(sender, e)
    decrease chosen month by one month
    LoadChart()
END FUNC
```

2.3.2 Cart form

2.3.2.1 Form screen shot



2.3.2.2 Structure chart



2.3.2.3 Pseudo code

```

FUNC frmCart_FormClosed()
    IF (frmMain is open)
        frmMain.frmCartOpen ← FALSE
    END IF
END FUNC

FUNC LoadDetails(userID)
    currentUserID ← userID
    dr ← "SELECT ProductName, Quantity, DiameterInMM, Material, tblCart.LengthInM,
    CartID, AmountInStock, PricePerMeter, tblCart.ProductID FROM tblCart, tblProduct WHERE
    UserID = " + userID + " AND tblCart.ProductID = tblProduct.ProductID ORDER BY Quantity DESC"
    heightMultiplier ← 1
    WHILE (dr.Read())
        every field is added to its own list
        heightMultiplier ← heightMultiplier + 1
    END WHILE
    IF (heightMultiplier == 1)
        emptyCart ← TRUE
    END IF
    height of Cart form = 150 + 60 * heightMultiplier
    LoadControls()
END FUNC

FUNC LoadControls()
    IF (emptyCart)
        create label that says: "You have not added any products to the cart..."
    END IF
    FOR each material in materialLst
        pull picture depending on material name
    create outline for picture
    END FOR
    FOR each product in productNameLst
        create label which text includes: ProductName, DiameterInMM, material
    END FOR
    FOR each qty in qtyLst
        create numericUpDown
        its maximum ← (stockLst[i] * 12) / lengthLst[i]
        its value ← qty
    END FOR
    FOR each length in lengthLst
        create label that says product's length
    END FOR
    FOR each item in productNameLst
        create pictureBox with "trash" icon for users to be able to delete products
    END FOR
    IF (!emptyCart)
        LoadPrices()
    END IF
END FUNC

```

```
FUNC PicTrash_Click()
    confirmation asked: "Are you sure that you want to remove that product from the cart?"
    IF (confirmed)
        i ← (sender as PictureBox.Tag)
        DoDML("DELETE FROM tblCart WHERE CartID = " + cartIDLst[i])
        refresh form
    END IF
END FUNC

FUNC LoadPrices()
    FOR EACH price IN priceLst
        create label with price Ex VAT, that says: "£" + rounded to 2dp (price *
lengthLst[i] * qtyLst[i])
    END FOR
    FOR EACH price IN priceLst
        create label with price Inc VAT, that says: "£" + rounded to 2dp (1.2 * price *
lengthLst[i] * qtyLst[i])
    END FOR
    create label total
    create label subtotal
    UploadTotals()
    create button checkout
END FUNC

FUNC btnCheckout_Click()
    IF (NOT frmCheckoutOpen)
        frmCheckoutOpen ← TRUE
        show frmCheckout
    ELSE
        bring form to front
    END IF
END FUNC

FUNC UploadTotals()
    total ← 0
    FOR EACH singlePrice IN priceLst
        total ← total + (singlePrice * lengthLst[i] * qtyLst[i])
    END FOR
    update labels
END FUNC

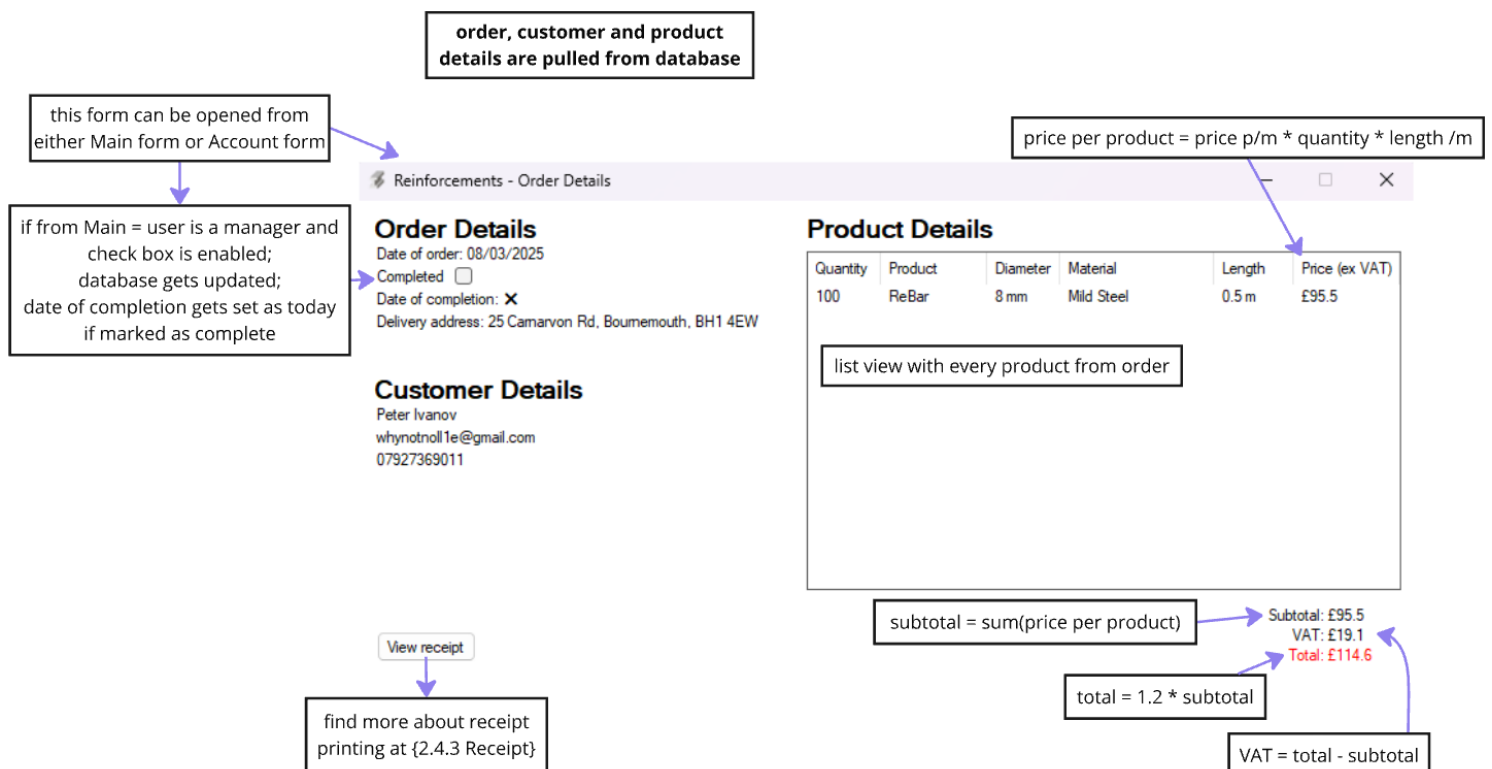
FUNC qtyPicker_ValueChanged()
    i ← (sender as NumericUpDown).Tag
    DoDML("UPDATE tblCart SET Quantity = " + (sender as NumericUpDown).Value + "
WHERE CartID = " + cartIDLst[i])
    update qty list
    update prices
    UploadTotals()
END FUNC
```

```

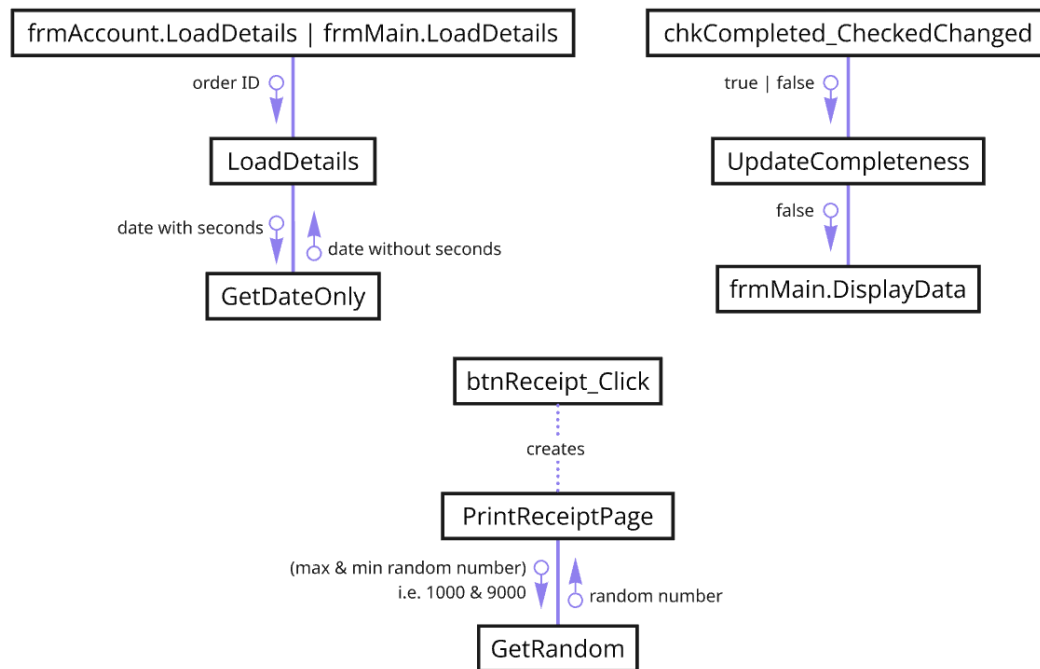
FUNC OrderCompleted(addressID)
    dr ← "DELETE FROM tblCart WHERE UserID = " + currentUserID
    DoDML("INSERT INTO tblOrder (UserID, DateOfOrder, TotalPaid, AddressID) " +
        "VALUES ('" + currentUserID + "', '" + DateTime.Now.ToString("dd/MM/yyyy") + "', '" +
        Math.Round(1.2 * total, 2) + "', '" + addressID + "')"
    dr ← "SELECT MAX(OrderID) FROM tblOrder"
    dr.Read()
    orderID ← MAX(OrderID)
    FOR EACH productID IN productIDLst
        DoDML("INSERT INTO tblProductOrder (OrderID, ProductID, Quantity,
LengthInM) " +
            "VALUES ('" + orderID + "', '" + productID + "', '" + qtyLst[i] + "', '" + lengthLst[i] + "')"
        END FOR
        piecesPerBar ← 12 / lengthLst[i]
        IF (piecesPerBar == 0)
            piecesPerBar ← 1
        END IF
        FOR EACH productID IN productIDLst
            DoDML("UPDATE tblProduct SET AmountInStock = AmountInStock - (" +
                Math.Ceiling((double)qtyLst[i] / piecesPerBar) + ") WHERE ProductID = " + productID)
            END FOR
    END FOR
END FUNC
    
```

2.3.3 Order Details form

2.3.3.1 Form screen shot



2.3.3.2 Structure chart



2.3.3.3 Pseudo code

```

FUNC LoadDetails(selectedOrderID)
    dr ← "SELECT tblOrder.UserID, tblOrder.DateOfOrder, tblOrder.TotalPaid,
tblOrder.Completed, tblOrder.DateOfCompletion, " +
        "(tblAddress.FirstLine & ', ' & tblAddress.Town & ', ' & tblAddress.Postcode) AS address "
    +
        "FROM (tblOrder INNER JOIN tblAddress ON tblOrder.AddressID = tblAddress.AddressID)
    " +
        "WHERE tblOrder.OrderID = " + selectedOrderID
    dr.Read()
    dateOfOrder ← frmMain.GetDateOnly(tblOrder.DateOfOrder) //view pseudo for Main
form for more info
    update dateOfOrder label
    subtotal ← tblOrder.TotalPaid / 1.2
    total ← tblOrder.TotalPaid
    VAT ← total - subtotal
    update text of label subtotal, VAT and total using rounding to 2dp
    IF (order is completed)
        checkbox is checked
    END IF
    label with dateOfCompletion ← frmMain.GetDateOnly(tblOrder.DateOfCompletion) //
view pseudo for Main form for more info
    update delivery address label
    dr ← "SELECT Quantity, ProductName, DiameterInMM, Material, PricePerMeter,
LengthInM FROM tblProductOrder, tblProduct " +
        "WHERE OrderID = " + selectedOrderID + " AND tblProductOrder.ProductID =
tblProduct.ProductID ORDER BY Quantity DESC"
  
```

```

        WHILE (dr.Read())
            add every product in order in list view
        END WHILE
        dr ← "SELECT FirstName, Surname, Email, PhoneNumber, CompanyName FROM tblUser
WHERE UserID = " + userID
        dr.Read()
        update labels with user details
        chkCompletedEvent ← true
    END FUNC

    FUNC chkCompleted_CheckedChanged()
        IF (chkCompletedEvent)
            IF (check box checked)
                confirmation asked: "Mark order #" + orderID + " as complete?"
                IF (confirmed)
                    UpdateCompleteness(true)
                ELSE
                    chkCompletedEvent ← false
                    set check box as unchecked
                END IF
            ELSE
                confirmation asked: "Mark order #" + orderID + " as incomplete?"
                IF (confirmed)
                    UpdateCompleteness(false)
                ELSE
                    chkCompletedEvent ← false
                    set check box as checked
                END IF
            END IF
        ELSE
            chkCompletedEvent ← true
        END IF
    END FUNC

    FUNC UpdateCompleteness(completed)
        cmdStr ← "UPDATE tblOrder SET Completed = " + completed + ", "
        IF (completed)
            today ← DateTime.Now.ToString("dd/MM/yyyy")
            cmdStr ← cmdStr + "DateOfCompletion = '" + today + "'"
        ELSE
            cmdStr ← cmdStr + "DateOfCompletion = null"
        END IF
        cmdStr ← cmdStr + " WHERE OrderID = " + orderID
        DoDML(cmdStr)
        refresh Main form
    END FUNC

```



```
FUNC btnReceipt_Click()
    receiptLength ← 450
    FOR EACH item IN productList
        receiptLength ← receiptLength + 15
    END FOR
    receipt ← new PrintDocument
    size of receipt ← (350, receiptLength)
    set printPreviewDialog1.Document ← receipt
    show printPreviewDialog1 dialog
    dispose receipt
END FUNC

FUNC PrintReceiptPage()
    draw store title
    draw store address and phone number
    draw order number, date of order, and masked card number
    draw column headers: NAME, QTY, PRICE

    x-coordinate set to default
    y-coordinate set to default
    FOR EACH product IN productList
        draw product at (x,y)
        y-coordinate increased
    END FOR

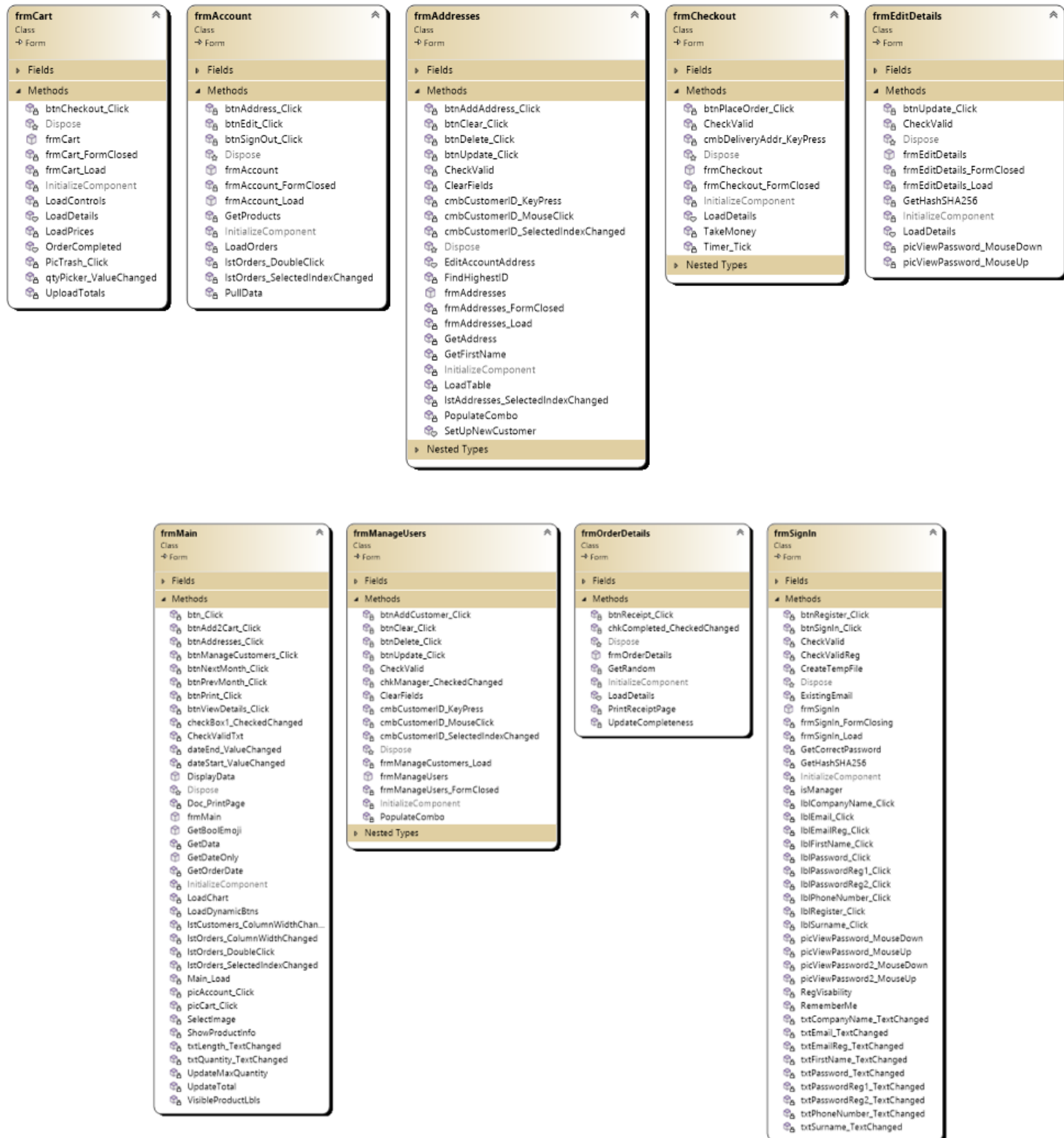
    x-coordinate increased
    y-coordinate set to default
    FOR EACH qty IN qtyList
        draw qty at (x, y)
        y-coordinate increased
    END FOR

    x-coordinate increased
    y-coordinate set to default
    FOR EACH price IN priceList
        draw price at (x, y)
        y-coordinate increased
    END FOR

    draw "SUBTOTAL", "VAT", "TOTAL"
    calculate and draw values for previous variables
    draw barcode
    draw "THANK YOU!"
END FUNC

FUNC GetRandom(min, max)
    return random integer between min and max
END FUNC
```

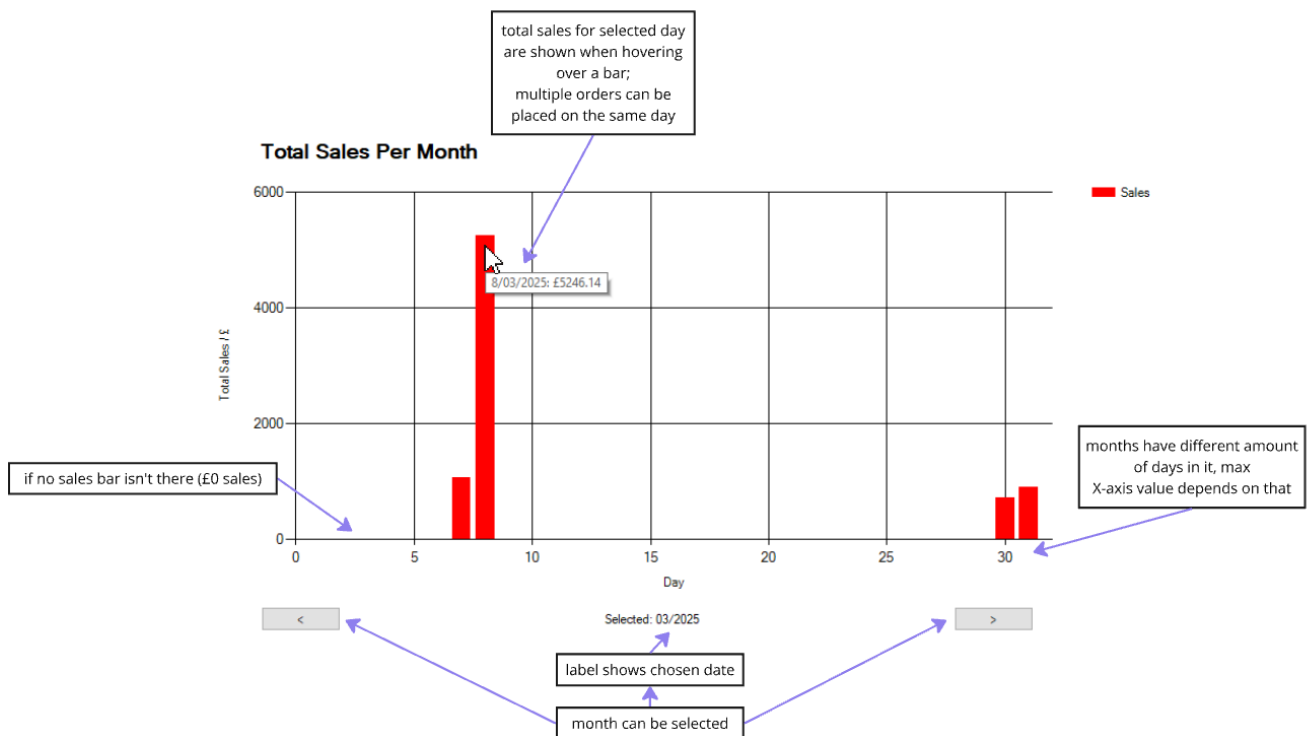
2.3.4 Class diagram (methods list)



2.4 Report design

2.4.1 Sales chart

2.4.1.1 Screen shot



2.4.1.2 Pseudo code

Function LoadChart() is called during Main_Load() and is responsible for loading the sales chart.

```

LoadChart()
    daysInMonth ← get number of days in selectedMonth
    salesData ← new dictionary

    FOR i ← 1 to daysInMonth
        salesData[i] ← 0
    END FOR

    clear chart series
    add new series "Sales" to chart

    IF (selectedMonth.Month < 10)
        lblDate.Text ← "Selected: 0M/YYYY"
        set chart tooltip format ← "#VALX/0M/YYYY: £#VALY"
    ELSE
        lblDate.Text ← "Selected: M/YYYY"
        set chart tooltip format ← "#VALX/M/YYYY: £#VALY"
    END IF
    
```

```

dr ← SELECT DAY(DateOfOrder), SUM(TotalPaid)
      FROM tblOrder
      WHERE YEAR(DateOfOrder) = 'selectedMonth.Year'
      AND MONTH(DateOfOrder) = 'selectedMonth.Month'
      GROUP BY DAY(DateOfOrder)
      ORDER BY DAY(DateOfOrder)

WHILE (dr.Read())
    orderDay ← dr[0]
    totalPaid ← dr[1]
    salesData[orderDay] ← totalPaid
END WHILE

FOR EACH entry in salesData
    add bar to chart with (X-value ← entry.Key) and (Y-value ← entry.Value)
END FOR

END

```

2.4.2 Order report

2.4.2.1 Screen shot

report can be printed/saved as pdf

number of pages depends on amount of data to print

date pickers, order report will include orders between these dates inclusive

print preview form is generated and opened

these controls are on Main form (statistics tab)

Reinforcements - Print Preview

ORDER REPORT
Between 08/11/2024 and 31/03/2025 inclusive

Order ID	Customer	Date	Paid
1	Peter, Ivanov	08/11/2024	£1021.8
3	Peter, Ivanov	07/03/2025	£1080
6	Peter, Ivanov	08/03/2025	£114.6
5	Peter, Ivanov	08/03/2025	£5131.54
8	Stepan, Stepanini	30/03/2025	£719.1
10	Peter, Ivanov	31/03/2025	£325.32
9	Dimon, Boegolovka	31/03/2025	£573
Total Sales:			£8965.36

Order Report
Start: 08/11/2024
End: 31/03/2025
Print

2.4.2.2 Pseudo code

```

btnPrint_Click()
    doc ← new PrintDocument
    header ← string with columns: Order ID, Customer, Date, Paid
    stringToPrint ← GetData()
    documentContents ← stringToPrint
    printPreviewDialog1.Document = doc
    show printPreviewDialog1
END

GetData()
    totalSales ← 0
    dataToPrint ← ""
    dr ← SELECT OrderID, FirstName, Surname, DateOfOrder, TotalPaid
        FROM tblOrder, tblUser
        WHERE tblOrder.UserID = tblUser.UserID
        AND tblOrder.DateOfOrder >= startDate AND tblOrder.DateOfOrder <= endDate
        ORDER BY DateOfOrder
    dataToPrint ← header

    WHILE (dr.Read())
        totalSales ← totalSales + dr[4]
        append formatted line to dataToPrint with OrderID, Customer Name,
        Date, and TotalPaid
    END WHILE

    IF (dataToPrint = header)
        append "NO DATA FOUND!" to dataToPrint
    ELSE
        append "Total Sales: £" + totalSales to dataToPrint
    END IF

    RETURN dataToPrint
END

Doc_PrintPage()
    draw title text "REINFORCEMENTS: ORDERS REPORT"
    draw date range text

    measure stringToPrint to get charactersOnPage and linesPerPage
    draw stringToPrint

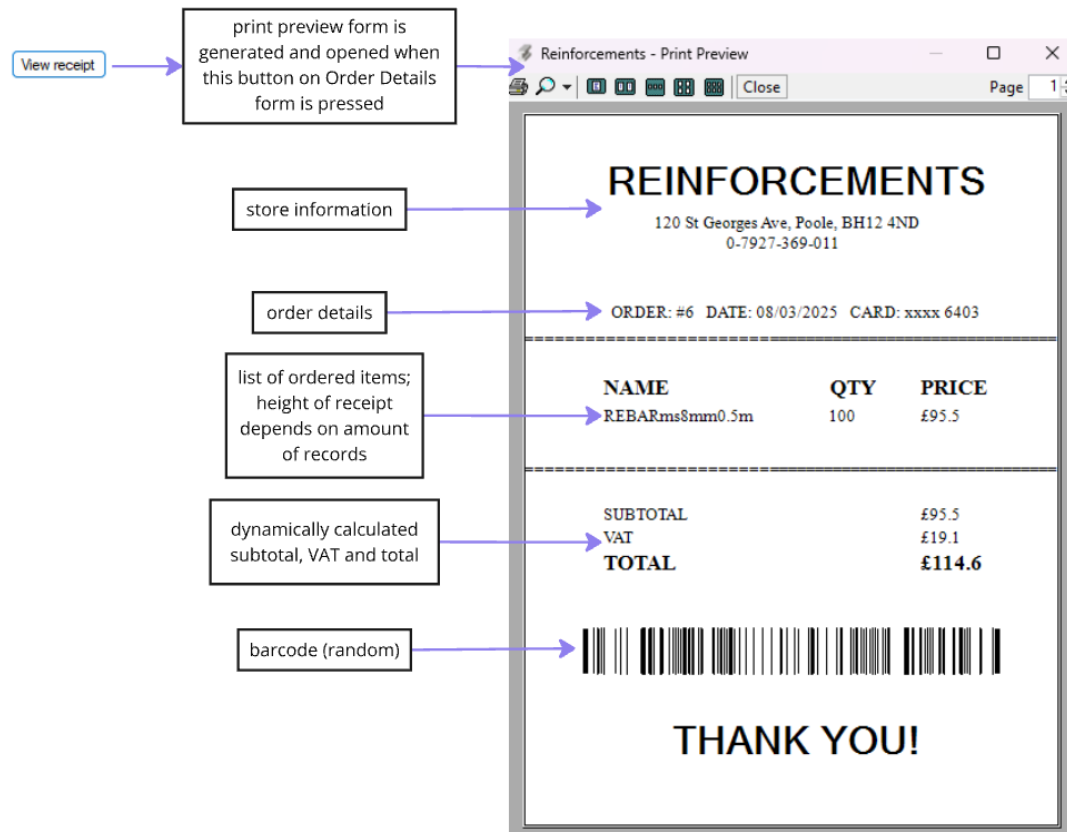
    stringToPrint ← remaining part after charactersOnPage

    IF (stringToPrint is not empty)
        stringToPrint ← documentContents
    ELSE
        stringToPrint ← header + stringToPrint
    END IF
END

```

2.4.3 Receipt

2.4.3.1 Screen shot



2.4.3.2 Pseudo code

```

btnReceipt_Click()
    receiptLength ← 450
    FOR EACH item IN productsList
        receiptLength ← receiptLength + 15
    END FOR

    receipt ← new PrintDocument
    size of receipt ← (350, receiptLength)
    set printPreviewDialog1.Document ← receipt
    show printPreviewDialog1 dialog
    dispose receipt
END
    
```

```
PrintReceiptPage()
  draw store title
  draw store address and phone number
  draw order number, date of order, and masked card number
  draw column headers: NAME, QTY, PRICE

  x-coordinate set to default
  y-coordinate set to default
  FOR EACH product IN productList
    draw product at (x,y)
    y-coordinate increased
  END FOR

  x-coordinate increased
  y-coordinate set to default
  FOR EACH qty IN qtyList
    draw qty at (x, y)
    y-coordinate increased
  END FOR

  x-coordinate increased
  y-coordinate set to default
  FOR EACH price IN priceList
    draw price at (x, y)
    y-coordinate increased
  END FOR

  draw "SUBTOTAL", "VAT", "TOTAL"
  calculate and draw values for previous variables
  draw barcode
  draw "THANK YOU!"

END
```

3. Testing

3.1 Test plan

No.	Area tested	Test data	Description purpose	Expected outcome	Output reference
1)	signing in	enter e-mail and password	checks if user with that e-mail exists and if password is correct	Main form should load if both fields match their regex; error message should be given if invalid input	Pass
2)	'remember me' button	tick 'remember me' check box	user doesn't have to log in after restarting the program	logged in user's e-mail should be saved in 'user details.txt'; software should start without asking user to log in	Pass
3)	registration	enter first name, surname, e-mail, phone number, company name, and password with its confirmation	checks if user with that e-mail exists and checks if other fields are valid	2 users cannot have same e-mail address; all fields should match their regex; error message should be given if invalid input; company name should be optional; password should be confirmed correctly; upon registering information should get stored in database; hashed version of password should be stored	Pass

4)	revealing password	press on a button	password is hidden by default for security reasons	corresponding password field should become unmasked while pressing and being over the button; during registration only 1 out of 2 passwords is revealed to force user remember password better	Pass
5)	view account	press on a button	Account form is opened; logged in user information and their orders are shown	only one form can be open simultaneously; correct data should be pulled; corresponding label becomes visible if no orders found by logged in user	Pass
6)	editing account details	press on a button then change details	Edit Details form is opened where user's details can be updated	only one form can be open simultaneously; correct data should be pulled initially; fields should match their regex; if password field is empty, it shouldn't get updated; reveal password button should work correctly (point 4); upon updating details Main form should refresh, so if logged in user is a manager and is viewing Users tab, changes should take place; database should be updated correctly; error message should be given if invalid input	Pass

7)	editing account addresses	press on a button then add/delete/update address	Address form is opened; list view is filled with previously added addresses (if any)	only one form can be open simultaneously; selected user cannot be changed, because responsible for changing <u>account</u> addresses; correct data should be pulled initially; pressing on a record from list view fills the text boxes with data; fields should match their regex; 'Clear' should empty the text boxes and deselect a record; database should get updated; error message should be given if invalid input	Pass
8)	signing out	press on a button	program is restarted; signing in is needed again	confirmation needed; program should restart without errors; 'user details.txt' should get deleted	Pass
9)	order details (Account form)	double-click on a record	Order Details form is opened; order, customer and product details are displayed; subtotal, VAT and total are calculated and shown; receipt can be viewed	only one form can be open simultaneously; correct data should be pulled initially; check box 'Completed' should be disabled, as customers aren't allowed to change that; subtotal, VAT and total should be calculated correctly	Semi-pass

10)	view receipt	press on a button	Print Preview Dialog form is opened; order and product details are displayed	height of receipt should be depending on the size of ordered products list; correct data should be pulled; subtotal, VAT and total should be calculated correctly	Pass
11)	view cart	press on a button	Cart form is opened; each product added to cart is shown	only one form can be opened simultaneously; product record should be created dynamically using correct data from database; total should be shown and correct	Pass
12)	changed quantity of product in cart	use numeric-up-down	users must be able to change the desired amount of product	user cannot order more than available in stock; prices should change by correct amount; database should get updated	Semi-pass
13)	deleting product from cart	press on a button	users must be able to delete products from cart easily, in case of putting wrong product in cart or changing their mind	confirmation should be asked; record of product should be deleted from the menu and database; label should appear if no products left in the cart after deleting	Pass
14)	view checkout	press on a button	Checkout form is opened	only one form can be opened simultaneously; all details should be shown correctly; combo box should be filled with user's addresses from database	Pass

15)	placing order	choose delivery address and enter payment details	order is placed if payment details are valid	order should be added to database; cart should get emptied; forms should refresh; fields should match their regex; card cannot be expired	Pass
16)	printing order report	choose dates and press on a button	Print Preview Dialog form is opened; order details are displayed between chosen dates	orders should be pulled correctly from database between chosen dates; 'NO DATA FOUND!' if no orders; total should be calculated correctly	Pass
17)	view total sales chart	choose Statistics tab on Main form	chart is loaded	data should be pulled correctly from database; sum of totals should be shown if multiple orders on the same day; bar height = 0 if no sales on a day	Pass
18)	change selected month for chart	press on a button	chart is reloaded for selected month	each month has either 28/29/30/31 days, correct number of days should be displayed; label that shows selected month should change correctly	Pass

19)	adding product to cart	choose Products tab on Main form, then enter values into fields and press on a button	data about product is displayed; length and quantity can be chosen; total is displayed	data about product should be pulled correctly from database; length can be between 300 and 6000 millimetres; quantity depends on length and amount of product in stock; corresponding error message should be shown if invalid inputs in fields; cannot add product if it's out of stock	Pass
20)	viewing managers/customers	choose Users tab on Main form, tick 'Managers' check box	managers normally don't need information about other managers, so they are separated in the Users list view	correct separation depending on the database	Pass
21)	editing user's details	choose a user and enter changes, then press clear/add/update/delete	Manage Users form is opened; managers can change users' details	only one form can be opened simultaneously; list view and text boxes should be filled with correct data from database; setting user as manager requires confirmation; 'Clear' button restarts the form; other buttons should do corresponding changes to database; Main form is refreshed after changes taking place	Pass

3.2 Test results

2 videos are recorded and uploaded as separate files:
5270_testing_part1.mkv and 5270_testing_part2.mkv

4. Evaluation

4.1 Objectives comparison

1. “Either log in or register” – Done in Sign In form, all sub-objectives are met, except implementation of ‘forgot password’ due to its complexity.
2. “Main form for managers will include statistics and two tables: orders and users. Both customers and managers can order products.” – Done in Main form, all sub-objectives are met.
3. “Order details can be viewed.” – Done in Order Details form, all sub-objectives are met.
4. “Users can be added/deleted/updated.” – Done in Manage Users form, all sub-objectives are met.
5. “Addresses can be added/deleted/updated.” – Done in Addresses form, all sub-objectives are met.
6. “Personal account management.” – Done in Account form, all sub-objectives are met.
7. “Cart which displays its contents.” – Done in Cart form, all sub-objectives are met.
8. “Checkout with summary, delivery address, and payment details.” – Done in Checkout form, all sub-objectives are met.

4.2 Improvements

1. Implementation of deadlines and order prioritisation would be very useful for the construction team.
2. Button ‘forgot password’ could be added using e-mail/SMS verification. It would be very useful for users as they don’t have to contact managers to reset password.
3. Ideally, length of products could be changed in Cart form. Doing so makes it easier for users to change length, without having to choose a product again with new desired length.
4. Some of rebars require bending in desired by customers ways. This feature could be added for better representation of business.
5. Barcode and card number shown in receipt are generated randomly. Both could be representing useful data.

4.3 Analysis of 3rd party feedback

After my program was complete, I have sent a copy to the manager of the business where I was doing my part-time job.

“The program developed for my company demonstrates a wide range of key features required for day-to-day operations, such as order and user management, reporting. From a management perspective, the interface is functional and aligns well with what our team needs. The ability to view orders, generate receipts, and monitor monthly sales is especially valuable. While there are some areas that could benefit from a more refined design or clearer layout, the core functionality is there and works reliably in most scenarios. Occasionally, the system behaves in ways that may be confusing to new users, so adding tooltips or clearer labels could be helpful. There is also potential to improve the system’s flexibility, such as offering more ways to filter or sort data. Overall, this is a very promising application that clearly had a lot of work put into it, and with a few usability and performance tweaks, it could be very effective for our business.” — Adam Smith, manager of Cumberland Reinforcements Ltd.

I was glad to hear that my program has potential to become useful in real-life business. However, I think the feedback is too generous, as my software clearly lacks some key features, such as deadline implementation, ability to design rebars for bending in desired ways. These things could be done and with a bit of polishing I believe they could implement my system in their workflow.

5. Technical solution

5.1 Class Main

frmMain (orders tab)

Reinforcements - Main Menu

Orders	Users	Products	Statistics			
Order ID	User ID	Date of Order	Total Paid	Completed	Date of Completion	Address ID
9	51	31/03/2025	£573	✗	✗	38
8	52	30/03/2025	£719.1	✗	✗	37
6	27	08/03/2025	£114.6	✗	✗	27
5	27	08/03/2025	£5131.54	✓	14/03/2025	19
3	27	07/03/2025	£1080	✓	12/03/2025	27
1	27	08/11/2024	£1021.8	✓	11/11/2024	19

View details

frmMain (users tab)

Reinforcements - Main Menu

Orders	Users	Products	Statistics		
User ID	First Name	Surname	Email	Phone Number	Company Name
52	Stepan	Stepanini	stepan1337@yahoo.com	07454345678	Pomidor Inc
51	Dimon	Boegolovka	dmon@gmail.com	07945345678	Dima Comp
50	Vladislav	Vlasov	vladvlasov@outlook.com	07936456789	Shelby Home
49	Samuel Ent...	McMillan	poopfart@outlook.com	07694835324	poop fart ltd
48	John Wick	Wick	b556986575489@gmail.com	0798975475	John wick business
47	Barack	Obunga	lovepeterivanov@yandex.ru	07597652361	The White House A few Years ago
45	Lxxx	Loxatron	lox122@yandex.ru	05050505552	loxcompany.ru
44	Dominico	Torreto	dominictorretto@gmail.com	07777777777	
39	Roman	Prhoda	roma1337@gmail.com	07345678912	Roma Comp
3	Boris	Johnson	bortisverygood@yahoo.com	03487623486	United Kingdom Inc
2	Ryan	Gosling	brokeman34842@outlook.com	07389275472	
1	Gerard	Sports	gerardsports1332@gmail.com	07463294014	Gerard Ltd

☐ Managers

Manage

Addresses

frmMain (products tab)

Reinforcements - Main Menu

OrdersUsersProductsStatistics

ReBar 8mm Stainless Steel£8.67

ReBar 8mm Mild Steel£1.91

ReBar 10mm Stainless Steel£9.84

ReBar 10mm Mild Steel£2.49

ReBar 12mm Stainless Steel£18.38

ReBar 12mm Mild Steel£3.44

ReBar 14mm Stainless Steel£27.11

ReBar 16mm Mild Steel£4.95
UNAVLIABLE

ReBar 18mm Stainless Steel£36

ReBar 20mm Mild Steel£7.99

ReBar 8mm Mild Steel£1.91

Custom length: 1000 mm (300 - 6000)

Quantity: 1 (1 - 5280)

Total: £2.29

Add to cart

ReBar 8mm Mild Steel£1.91

Custom length: 1000 mm (300 - 6000)

Quantity: 1 (1 - 5280)

Total: £2.29

Add to cart

frmMain (statistics tab)

Reinforcements - Main Menu

OrdersUsersProductsStatistics

Order Report

Start: 08/11/2024

End: 31/03/2025

Print

Total Sales Per Month

Total Sales £

Day

Sales

<

Selected: 03/2025

>

C# code

```
namespace Coursework
{
    public partial class frmMain : Form
    {
        clsDBConnector dbConnector = new clsDBConnector();
        OleDbDataReader dr;
        string sqlStr;

        public bool frmMUOpen = false;
        public bool frmAddrOpen = false;
        public bool frmAccOpen = false;
        public bool frmCartOpen = false;

        private frmManageUsers frmManageUsers = null;
        private frmAddresses frmAddresses = null;
        private frmAccount frmAccount = null;
        private frmCart frmCart = null;

        private const int MaxColumnWidth = 200;
        string selectedOrderID = "";

        private string documentContents;
        private string stringToPrint;
        private string header;
        private string startDate;
        private string endDate;
        private DateTime selectedMonth = DateTime.Now;

        private int productInStock;
        private string productID;
        private string userID;
        private bool manager = false;

        public frmMain()
        {
            InitializeComponent();
        }

        private void Main_Load(object sender, EventArgs e)
        {
            DisplayData(false);
            if (!manager)
            {
                tabControl1.TabPages.Remove(tabPage1);
                tabControl1.TabPages.Remove(tabPage2);
                tabControl1.TabPages.Remove(tabPage4);
            }
            VisibleProductLbls(false);
            LoadDynamicBtns();
            LoadChart();
            dateStart.Value = GetOrderDate("MIN");
            dateEnd.Value = GetOrderDate("MAX");
        }

        private void LoadChart()
        {
            int daysInMonth = DateTime.DaysInMonth(selectedMonth.Year, selectedMonth.Month);
            Dictionary<int, double> salesData = new Dictionary<int, double>();
            for (int i = 1; i <= daysInMonth; i++)
            {
                salesData[i] = 0;
            }
            chartSales.Series.Clear();
            chartSales.Series.Add("Sales");
            chartSales.Series["Sales"].Color = Color.Red;
            chartSales.ChartAreas[0].AxisX.Title = "Day";
            chartSales.ChartAreas[0].AxisY.Title = "Total Sales / £";
            chartSales.BorderWidth = 2;
        }
    }
}
```

```

        if ((int)selectedMonth.Month < 10)
        {
            lblDate.Text = $"Selected: 0{selectedMonth.Month}/{selectedMonth.Year}";
            chartSales.Series["Sales"].ToolTip =
            $"#VALX/0{selectedMonth.Month}/{selectedMonth.Year}: £#VALY";
        }
        else
        {
            lblDate.Text = $"Selected: {selectedMonth.Month}/{selectedMonth.Year}";
            chartSales.Series["Sales"].ToolTip =
            $"#VALX/{selectedMonth.Month}/{selectedMonth.Year}: £#VALY";
        }

        dbConnector.Connect();
        sqlStr = "SELECT DAY(DateOfOrder), SUM(TotalPaid) " +
            "FROM tblOrder " +
            $"WHERE YEAR(DateOfOrder) = '{selectedMonth.Year}' AND MONTH(DateOfOrder) = '{selectedMonth.Month}' " +
            "GROUP BY DAY(DateOfOrder) ORDER BY DAY(DateOfOrder)";
        dr = dbConnector.DoSQL(sqlStr);
        while (dr.Read())
        {
            int orderDay = Convert.ToInt32(dr[0]);
            double totalPaid = Convert.ToDouble(dr[1]);
            salesData[orderDay] = totalPaid;
        }
        dbConnector.Close();
        foreach (var entry in salesData)
        {
            chartSales.Series["Sales"].Points.AddXY(entry.Key, entry.Value);
        }
    }

    private void VisibleProductLbls(bool v)
    {
        lblProductName.Visible = v;
        lblDiameter.Visible = v;
        lblMaterial.Visible = v;
        lblPricePM.Visible = v;
        lblPriceVAT.Visible = v;
        lblTotal.Visible = v;
        lblStock.Visible = v;
        lblCstLength.Visible = v;
        txtLength.Visible = v;
        lblLengthMeters.Visible = v;
        lblQuantity.Visible = v;
        txtQuantity.Visible = v;
        lblValidLength.Visible = v;
        lblValidQuantity.Visible = v;
        btnAdd2Cart.Visible = v;
    }

    private void LoadDynamicBtns()
    {
        int i = 0;
        int y = 10;
        int x = 10;
        dbConnector.Connect();
        sqlStr = "SELECT ProductID, ProductName, DiameterInMM, Material, PricePerMeter, AmountInStock FROM tblProduct ORDER BY DiameterInMM";
        dr = dbConnector.DoSQL(sqlStr);
        while (dr.Read())
        {
            Button btn = new Button();
            btn.BackColor = Color.Transparent;
            btn.ForeColor = Color.LimeGreen;
            if (Convert.ToInt32(dr[5]) == 0)
            {
                btn.ForeColor = Color.Red;
            }
            btn.Size = new Size(100, 100);
        }
    }

```

```

        if (x > 1000)
        {
            y += 100;
            x = 10;
        }
        btn.Location = new Point(x, y);
        btn.Visible = true;
        btn.Tag = dr[0].ToString();
        Font font = new Font("Microsoft Sans Serif", 8.25f, FontStyle.Bold);
        btn.Font = font;
        btn.Text =
        $"{dr[1].ToString()}\n{dr[2].ToString()}mm\n{dr[3].ToString()}\n£{dr[4].ToString()}";
        if (Convert.ToInt32(dr[5]) == 0)
        {
            btn.Text += "\nUNAVLIABLE";
        }
        btn.Name = "btn_ " + i;
        i++;
        x += 110;
        btn.Image = SelectImage(dr[1].ToString(), dr[3].ToString());
        btn.Click += btn_Click;
        tabPage3.Controls.Add(btn);
    }
    dbConnector.Close();
}

private void btn_Click(object sender, EventArgs e)
{
    VisibleProductLbls(true);
    ShowProductInfo((sender as Button).Tag.ToString());
}

double price;

private void ShowProductInfo(string productid)
{
    productID = productid;
    dbConnector.Connect();
    sqlStr = $"SELECT ProductName, DiameterInMM, Material, PricePerMeter, AmountInStock
FROM tblProduct WHERE ProductID = {productID}";
    dr = dbConnector.DoSQL(sqlStr);
    dr.Read();
    lblProductName.Text = dr[0].ToString();
    lblDiameter.Text = dr[1].ToString() + "mm";
    lblMaterial.Text = dr[2].ToString();
    price = Convert.ToDouble(dr[3]);
    double priceVAT = Math.Round(price * 1.2, 2);
    lblPricePM.Text = $"£{price}";
    lblPriceVAT.Text = $"£{priceVAT} inc VAT";
    productInStock = Convert.ToInt32(dr[4]);
    lblStock.Text = $"Stock: {productInStock}";
    dbConnector.Close();
    txtLength.Text = "1000";
    txtQuantity.Text = "1";
    if (productInStock == 0)
    {
        btnAdd2Cart.Enabled = false;
        lblValidQuantity.ForeColor = Color.Red;
        lblValidQuantity.Text = "OUT OF STOCK!";
    }
    else
    {
        btnAdd2Cart.Enabled = true;
        lblValidQuantity.ForeColor = Color.Black;
        lblValidQuantity.Text = $"(1 - {productInStock * 12})";
    }
    lblTotal.Text = $"Total: £{priceVAT}";
    lblValidLength.ForeColor = Color.Black;
}

```

```

private Image SelectImage(string productName, string material)
{
    if (productName.ToLower() == "rebar")
    {
        if (material.ToLower() == "mild steel")
        {
            return Image.FromFile("rebar_mild_steel.png");
        }
        if (material.ToLower() == "stainless steel")
        {
            return Image.FromFile("rebar_stainless_steel.png");
        }
    }
    return null;
}

private DateTime GetOrderDate(string maxOrMin)
{
    dbConnector.Connect();
    sqlStr = $"SELECT {maxOrMin}(DateOfOrder) FROM tblOrder";
    dr = dbConnector.DoSQL(sqlStr);
    dr.Read();
    DateTime date = Convert.ToDateTime(dr[0]);
    dbConnector.Close();
    return date;
}

public string GetBoolEmoji(string boolean)
{
    if (boolean.ToLower() == "true")
    {
        return "✓";
    }
    return "✗";
}

public void DisplayData(bool v)
{
    try
    {
        StreamReader currentFile = new StreamReader("temp.txt");
        dbConnector.Connect();
        sqlStr = $"SELECT UserID, Manager FROM tblUser WHERE Email = '{currentFile.ReadLine()}'";
        currentFile.Close();
        dr = dbConnector.DoSQL(sqlStr);
        dr.Read();
        userID = dr[0].ToString();
        if (dr[1].ToString() == "True")
        {
            manager = true;
        }

        selectedOrderID = "";
        sqlStr = "SELECT OrderID, UserID, DateOfOrder, TotalPaid, Completed, DateOfCompletion, AddressID FROM tblOrder ORDER BY Completed DESC";
        dr = dbConnector.DoSQL(sqlStr);
        lstOrders.Items.Clear();
        while (dr.Read())
        {
            lstOrders.Items.Add(dr[0].ToString());
            lstOrders.Items[lstOrders.Items.Count-1].SubItems.Add(dr[1].ToString());
            lstOrders.Items[lstOrders.Items.Count - 1].SubItems.Add(GetDateOnly(dr[2]));
            lstOrders.Items[lstOrders.Items.Count - 1].SubItems.Add("£" +
dr[3].ToString());
            lstOrders.Items[lstOrders.Items.Count -
1].SubItems.Add(GetBoolEmoji(dr[4].ToString()));
            lstOrders.Items[lstOrders.Items.Count - 1].SubItems.Add(GetDateOnly(dr[5]));
            lstOrders.Items[lstOrders.Items.Count - 1].SubItems.Add(dr[6].ToString());
        }
    }
}

```

```

        sqlStr = $"SELECT UserID, FirstName, Surname, Email, PhoneNumber, CompanyName
FROM tblUser WHERE Manager = {v} ORDER BY UserID DESC";
        dr = dbConnector.DoSQL(sqlStr);
        lstCustomers.Items.Clear();
        while (dr.Read())
        {
            lstCustomers.Items.Add(dr[0].ToString());
            lstCustomers.Items[lstCustomers.Items.Count -
1].SubItems.Add(dr[1].ToString());
            lstCustomers.Items[lstCustomers.Items.Count -
1].SubItems.Add(dr[2].ToString());
            lstCustomers.Items[lstCustomers.Items.Count -
1].SubItems.Add(dr[3].ToString());
            lstCustomers.Items[lstCustomers.Items.Count -
1].SubItems.Add(dr[4].ToString());
            lstCustomers.Items[lstCustomers.Items.Count -
1].SubItems.Add(dr[5].ToString());
        }
        dbConnector.Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error occured: " + ex.Message, "Error");
    }
}

public string GetDateOnly(object originalDBDate)
{
    if (originalDBDate != DBNull.Value)
    {
        DateTime fullDate = Convert.ToDateTime(originalDBDate);
        return fullDate.ToString("dd/MM/yyyy");
    }
    return "X";
}

e) private void lstCustomers_ColumnWidthChanged(object sender, ColumnWidthChangedEventArgs
{
    if (lstCustomers.Columns[e.ColumnIndex].Width > MaxColumnWidth)
    {
        lstCustomers.Columns[e.ColumnIndex].Width = MaxColumnWidth;
    }
}

private void lstOrders_ColumnWidthChanged(object sender, ColumnWidthChangedEventArgs e)
{
    if (lstOrders.Columns[e.ColumnIndex].Width > MaxColumnWidth)
    {
        lstOrders.Columns[e.ColumnIndex].Width = MaxColumnWidth;
    }
}

private void btnManageCustomers_Click(object sender, EventArgs e)
{
    if (!frmMUOpen)
    {
        frmMUOpen = true;
        frmManageUsers = new frmManageUsers();
        frmManageUsers.Show();
    }
    else
    {
        frmManageUsers.BringToFront();
        frmManageUsers.WindowState = FormWindowState.Normal;
    }
}

```

```

private void btnAddresses_Click(object sender, EventArgs e)
{
    if (!frmAddrOpen)
    {
        frmAddrOpen = true;
        frmAddresses = new frmAddresses();
        frmAddresses.Show();
    }
    else
    {
        frmAddresses.BringToFront();
        frmAddresses.WindowState = FormWindowState.Normal;
    }
}

private void picAccount_Click(object sender, EventArgs e)
{
    if (!frmAccOpen)
    {
        frmAccOpen = true;
        frmAccount = new frmAccount();
        frmAccount.Show();
    }
    else
    {
        frmAccount.BringToFront();
        frmAccount.WindowState = FormWindowState.Normal;
    }
}

private void checkBox1_CheckedChanged(object sender, EventArgs e)
{
    if (checkBox1.Checked)
    {
        DisplayData(true);
    }
    else
    {
        DisplayData(false);
    }
}

private void lstOrders_SelectedIndexChanged(object sender, EventArgs e)
{
    if (lstOrders.SelectedItems.Count > 0)
    {
        ListViewItem selectedItem = lstOrders.SelectedItems[0];
        selectedOrderID = selectedItem.SubItems[0].Text;
    }
    else
    {
        selectedOrderID = "";
    }
}

private void btnViewDetails_Click(object sender, EventArgs e)
{
    if (selectedOrderID == "")
    {
        MessageBox.Show("You have not selected an order to view.", "Error");
    }
    else
    {
        frmOrderDetails frmOrderDetails = new frmOrderDetails();
        frmOrderDetails.Show();
        frmOrderDetails.LoadDetails(selectedOrderID);
    }
}

```



```

private void Doc_PrintPage(object sender, PrintPageEventArgs e)
{
    int charactersOnPage = 0;
    int linesPerPage = 0;
    Font myFont1 = new Font("Courier New", 12.0f);
    Font myFont2 = new Font("Cooper", 18, FontStyle.Bold);
    e.Graphics.DrawString("REINFORCEMENTS: ORDERS REPORT", myFont2, Brushes.Black, 170,
35);
    e.Graphics.DrawString($"Between {dateStart.Value.ToString("dd/MM/yyyy")} and
{dateEnd.Value.ToString("dd/MM/yyyy")} inclusive", myFont1, Brushes.Black, 190, 70);
    e.Graphics.MeasureString(stringToPrint, myFont1, e.MarginBounds.Size,
StringFormat.GenericTypographic, out charactersOnPage, out linesPerPage);
    e.Graphics.DrawString(stringToPrint, myFont1, Brushes.Black, e.MarginBounds,
StringFormat.GenericTypographic);
    stringToPrint = stringToPrint.Substring(charactersOnPage);
    e.HasMorePages = (stringToPrint.Length > 0);
    if (!e.HasMorePages)
    {
        stringToPrint = documentContents;
    }
    else
    {
        stringToPrint = header + stringToPrint;
    }
}

private void dateStart_ValueChanged(object sender, EventArgs e)
{
    startDate = dateStart.Value.ToString("#MM/dd/yyyy#");
}

private void dateEnd_ValueChanged(object sender, EventArgs e)
{
    endDate = dateEnd.Value.ToString("#MM/dd/yyyy#");
}

private bool CheckValidTxt(int max, string input)
{
    if (input == "length")
    {
        try
        {
            int length = Convert.ToInt32(txtLength.Text);
            if (length >= 300 && length <= max)
            {
                UpdateMaxQuantity();
                UpdateTotal();
                return true;
            }
        }
        catch (Exception)
        {
            return false;
        }
    }
    else if (input == "quantity")
    {
        try
        {
            int MMInStock = max * 12000;
            int maxQuantity = MMInStock / Convert.ToInt32(txtLength.Text);
            int quantity = Convert.ToInt32(txtQuantity.Text);
            if (quantity >= 1 && quantity <= maxQuantity)
            {
                UpdateTotal();
                return true;
            }
        }
        catch (Exception)
        {
            return false;
        }
    }
    return false;
}

```

```

double total;

private void UpdateTotal()
{
    double length = Convert.ToDouble(txtLength.Text);
    int quantity = Convert.ToInt32(txtQuantity.Text);
    total = Math.Round(1.2 * (price * (length / 1000) * quantity), 2);
    lblTotal.Text = $"Total: £{total}";
}

private void UpdateMaxQuantity()
{
    if (productInStock != 0)
    {
        int maxQuantity = (productInStock * 12000) / Convert.ToInt32(txtLength.Text);
        lblValidQuantity.Text = $"(1 - {maxQuantity})";
    }
}

private void txtLength_TextChanged(object sender, EventArgs e)
{
    if (!CheckValidTxt(6000, "length"))
    {
        lblValidLength.ForeColor = Color.Red;
    }
    else
    {
        lblValidLength.ForeColor = Color.Black;
    }
}

private void txtQuantity_TextChanged(object sender, EventArgs e)
{
    if (!CheckValidTxt(productInStock, "quantity"))
    {
        lblValidQuantity.ForeColor = Color.Red;
    }
    else
    {
        lblValidQuantity.ForeColor = Color.Black;
    }
}

private void btnAdd2Cart_Click(object sender, EventArgs e)
{
    if (CheckValidTxt(6000, "length") && CheckValidTxt(productInStock, "quantity"))
    {
        dbConnector.Connect();
        string cmdStr = $"INSERT INTO tblCart (UserID, ProductID, Quantity, LengthInM) "
+
        $"VALUES ('{userID}', '{productID}', '{txtQuantity.Text}',
'{{Convert.ToDouble(txtLength.Text) / 1000}}')";
        dbConnector.DoDML(cmdStr);
        dbConnector.Close();
        MessageBox.Show($"Successfully added to cart. You can edit the contents of your
cart anytime.", "Product added to cart");
        if (frmCartOpen == true)
        {
            frmCart.Close();
            frmCart = new frmCart();
            frmCart.Show();
            frmCart.LoadDetails(userID);
        }
    }
    else
    {
        MessageBox.Show("Invalid values for length/quantity or both.", "Invalid
values");
    }
}

private void btnNextMonth_Click(object sender, EventArgs e)
{
    selectedMonth = selectedMonth.AddMonths(1);
    LoadChart();
}

```

```
private void btnPrevMonth_Click(object sender, EventArgs e)
{
    selectedMonth = selectedMonth.AddMonths(-1);
    LoadChart();
}
}
```

5.2 Class Account

frmAccount

ID	Date of Order	Total Paid	Completed	Date of Completion	Address	Products
6	08/03/2025	£114.6	✗	✗	25 Camarvon Rd, Boumemouth, BH11 4EW	100 x ReBar (8mm, 0.5m, mild steel)
5	08/03/2025	£5131.54	✓	14/03/2025	2 Bohemia Road, Boumemouth, BH11 9FT	100 x ReBar (8mm, 2.5m, mild steel), 50 x ReBar (10mm, 1m, mild steel), 30 x ReBar (16mm, 3.6m, stainless steel), 10 x ReBar (12mm, 1m, stainless steel)
3	07/03/2025	£1080	✓	12/03/2025	25 Camarvon Rd, Boumemouth, BH11 4EW	25 x ReBar (20mm, 1m, stainless steel)
1	08/11/2024	£1021.8	✓	11/11/2024	2 Bohemia Road, Boumemouth, BH11 9FT	20 x ReBar (8mm, 1m, mild steel), 30 x ReBar (16mm, 1m, stainless steel)

C# code

```
namespace Coursework
{
    public partial class frmAccount : Form
    {
        public frmAccount()
        {
            InitializeComponent();
        }

        string selectedOrderID = "";
        string userID;
        string email;
        string firstName;
        string surname;
        string phoneNumber;
        string companyName;
        bool manager = false;
        bool ordered = false;

        public bool frmEditOpen = false;
        public bool frmAddrOpen = false;

        private frmAddresses frmAddresses = null;
        private frmEditDetails frmEditDetails = null;

        public void frmAccount_Load(object sender, EventArgs e)
        {
            PullData();
            lblAccountType.Text = " Customer ";

            if (manager)
```

```

        {
            lblAccountType.Text = " Manager ";
        }
        lblName.Text = $"{firstName} {surname}";
        LoadOrders();
    }

    private void LoadOrders()
    {
        if (!ordered)
        {
            lstOrders.Visible = false;
        }
        else
        {
            clsDBConnector dbConnector = new clsDBConnector();
            OleDbDataReader dr;
            string sqlStr;
            dbConnector.Connect();
            sqlStr = "SELECT tblOrder.OrderID, tblOrder.DateOfOrder, tblOrder.TotalPaid,
tblOrder.Completed, tblOrder.DateOfCompletion, (tblAddress.FirstLine & ', ' & tblAddress.Town &
', ' & tblAddress.Postcode) AS address " +
                "FROM (tblOrder INNER JOIN tblAddress ON tblOrder.AddressID =
tblAddress.AddressID) " +
                $"WHERE tblOrder.UserID = {userID} " +
                "ORDER BY tblOrder.OrderID DESC";
            dr = dbConnector.DoSQL(sqlStr);
            lstOrders.Items.Clear();
            while (dr.Read())
            {
                lstOrders.Items.Add(dr[0].ToString());
                lstOrders.Items[lstOrders.Items.Count -
1].SubItems.Add((Application.OpenForms["frmMain"] as frmMain).GetDateOnly(dr[1]).ToString());
                lstOrders.Items[lstOrders.Items.Count - 1].SubItems.Add("£" +
dr[2].ToString());
                lstOrders.Items[lstOrders.Items.Count -
1].SubItems.Add((Application.OpenForms["frmMain"] as frmMain).GetBoolEmoji(dr[3].ToString()));
                lstOrders.Items[lstOrders.Items.Count -
1].SubItems.Add((Application.OpenForms["frmMain"] as frmMain).GetDateOnly(dr[4]).ToString());
                lstOrders.Items[lstOrders.Items.Count - 1].SubItems.Add(dr[5].ToString());
                lstOrders.Items[lstOrders.Items.Count -
1].SubItems.Add(GetProducts(Convert.ToInt32(dr[0])));
            }
            dbConnector.Close();
        }
    }

    private string GetProducts(int orderID)
    {
        string products = "";
        int count = 0;
        clsDBConnector dbConnector = new clsDBConnector();
        OleDbDataReader dr;
        string sqlStr;
        dbConnector.Connect();
        sqlStr = "SELECT Quantity, ProductName, DiameterInMM, Material, LengthInM FROM
tblProductOrder, tblProduct " +
                $"WHERE OrderID = {orderID} AND tblProductOrder.ProductID =
tblProduct.ProductID";
        dr = dbConnector.DoSQL(sqlStr);
        while (dr.Read())
        {
            if (count != 0)
            {
                products += ", ";
            }
            products += $"{dr[0]} x {dr[1]} ({dr[2]}mm, {dr[4]}m,
{dr[3].ToString().ToLower()})";
            count++;
        }
        dbConnector.Close();
        return products;
    }

    private void PullData()

```

```

{
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    StreamReader currentFile = new StreamReader("temp.txt");
    email = currentFile.ReadLine();
    currentFile.Close();
    dbConnector.Connect();
    sqlStr = $"SELECT FirstName, Surname, PhoneNumber, CompanyName, Manager, UserID FROM
tblUser WHERE Email = '{email}'";
    dr = dbConnector.DoSQL(sqlStr);
    while (dr.Read())
    {
        firstName = dr[0].ToString();
        surname = dr[1].ToString();
        phoneNumber = dr[2].ToString();
        companyName = dr[3].ToString();
        if (dr[4].ToString() == "True")
        {
            manager = true;
        }
        userID = dr[5].ToString();
    }

    sqlStr = $"SELECT OrderID FROM tblOrder WHERE UserID = {userID}";
    dr = dbConnector.DoSQL(sqlStr);
    dr.Read();
    try
    {
        if (dr[0] != DBNull.Value)
        {
            ordered = true;
        }
    }
    catch (Exception)
    {
    }
    dbConnector.Close();
}

private void btnSignOut_Click(object sender, EventArgs e)
{
    DialogResult dialogResult = MessageBox.Show($"Are you sure that you want to sign
out?", "Signing Out", MessageBoxButtons.YesNo);
    if (dialogResult == DialogResult.Yes)
    {
        File.Delete("user details.txt");
        File.Delete("temp.txt");
        Application.Restart();
    }
}

private void btnEdit_Click(object sender, EventArgs e)
{
    if (!frmEditOpen)
    {
        frmEditOpen = true;
        frmEditDetails = new frmEditDetails();
        frmEditDetails.Show();
        frmEditDetails.LoadDetails(userID, firstName, surname, email, phoneNumber,
companyName);
    }
    else
    {
        frmEditDetails.BringToFront();
        frmEditDetails.WindowState = FormWindowState.Normal;
    }
}

private void btnAddress_Click(object sender, EventArgs e)

```

```

    {
        if (!frmAddrOpen)
        {
            frmAddrOpen = true;
            frmAddresses = new frmAddresses();
            frmAddresses.Show();
            frmAddresses.EditAccountAddress(userID);
        }
        else
        {
            frmAddresses.BringToFront();
            frmAddresses.WindowState = FormWindowState.Normal;
        }
    }

    private void frmAccount_FormClosed(object sender, FormClosedEventArgs e)
    {
        if (Application.OpenForms["frmMain"] as frmMain != null)
        {
            (Application.OpenForms["frmMain"] as frmMain).frmAccOpen = false;
        }
    }

    private void lstOrders_SelectedIndexChanged(object sender, EventArgs e)
    {
        if (lstOrders.SelectedItems.Count > 0)
        {
            ListViewItem selectedItem = lstOrders.SelectedItems[0];
            selectedOrderID = selectedItem.SubItems[0].Text;
        }
        else
        {
            selectedOrderID = "";
        }
    }

    private void lstOrders_DoubleClick(object sender, EventArgs e)
    {
        if (selectedOrderID != "")
        {
            frmOrderDetails frmOrderDetails = new frmOrderDetails();
            frmOrderDetails.Show();
            frmOrderDetails.LoadDetails(selectedOrderID);
            frmOrderDetails.chkCompleted.Enabled = false;
        }
    }
}

```

5.3 Class Addresses

frmAddresses

Reinforcements - Address Management

Address Management

Select user: Ivanov, Peter

First line:

Town:

Postcode:

Clear Add Update Delete

ID	First Line	Town	Postcode
19	2 Bohemia Road	Bournemouth	BH11 9FT
27	25 Camarvon Rd	Bournemouth	BH1 4EW

C# code

```
namespace Coursework
{
    public partial class frmAddresses : Form
    {
        public frmAddresses()
        {
            InitializeComponent();
        }

        string selectedAddressID = "";
        bool altClearBtn = false;

        class CLsUser
        {
            public int userid { get; set; }
            public string username { get; set; }
        }

        private void frmAddresses_Load(object sender, EventArgs e)
        {
            cmbCustomerID.Text = "Select";
        }

        private void ClearFields()
        {
            txtFirstLine.Text = "";
            txtTown.Text = "";
            txtPostcode.Text = "";
        }

        private void cmbCustomerID_MouseClick(object sender, MouseEventArgs e)
        {
            if (cmbCustomerID.Text == "Select")
            {
                PopulateCombo();
            }
        }
    }
}
```

```

private void PopulateCombo()
{
    List<CLsUser> userList = new List<CLsUser>();
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    dbConnector.Connect();
    sqlStr = "SELECT UserID, (Surname & " + "','" + "& FirstName) as username FROM
tblUser";
    dr = dbConnector.DoSQL(sqlStr);
    while (dr.Read())
    {
        userList.Add(new CLsUser { userid = Convert.ToInt32(dr[0]), username =
dr[1].ToString() });
    }
    cmbCustomerID.DisplayMember = "username";
    cmbCustomerID.ValueMember = "userid";
    cmbCustomerID.DataSource = userList;
    dbConnector.Close();
}

private void cmbCustomerID_SelectedIndexChanged(object sender, EventArgs e)
{
    if (cmbCustomerID.SelectedValue != null)
    {
        LoadTable();
    }
}

private void LoadTable()
{
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    dbConnector.Connect();
    sqlStr = "SELECT AddressID, FirstLine, Town, Postcode" +
            " FROM tblAddress" +
            " WHERE UserID = " + cmbCustomerID.SelectedValue;
    dr = dbConnector.DoSQL(sqlStr);
    lstAddresses.Items.Clear();
    while (dr.Read())
    {
        lstAddresses.Items.Add(dr[0].ToString());
        lstAddresses.Items[lstAddresses.Items.Count - 1].SubItems.Add(dr[1].ToString());
        lstAddresses.Items[lstAddresses.Items.Count - 1].SubItems.Add(dr[2].ToString());
        lstAddresses.Items[lstAddresses.Items.Count - 1].SubItems.Add(dr[3].ToString());
    }
    dbConnector.Close();
    ClearFields();
    selectedAddressID = "";
}

private void btnAddAddress_Click(object sender, EventArgs e)
{
    if (cmbCustomerID.Text == "Select")
    {
        MessageBox.Show("You need to select a user first.", "Error");
    }
    else
    {
        if (CheckValid() == true)
        {
            clsDBConnector dbConnector = new clsDBConnector();
            string cmdStr = $"INSERT INTO tblAddress (UserID, FirstLine, Town, Postcode)
" +
                $"VALUES ({cmbCustomerID.SelectedValue},
'{txtFirstLine.Text}', '{txtTown.Text}', '{txtPostcode.Text}')";
            dbConnector.Connect();
            dbConnector.DoDML(cmdStr);
            dbConnector.Close();
            cmbCustomerID_SelectedIndexChanged(sender, e);
        }
    }
}

```



```

private bool CheckValid()
{
    if (txtFirstLine.Text == "" || txtTown.Text == "" || txtPostcode.Text == "")
    {
        MessageBox.Show("You have not entered anything for one or many fields.",
"Error");
        return false;
    }
    string pattern = @"^\d+[A-Za-z]?[s]([A-Z][a-z]*)([s]([A-Z][a-z]*))*";
    Match tryToMatch = Regex.Match(txtFirstLine.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid first line of address (e.g., 1a Alexander Road).",
"Error");
        return false;
    }
    pattern = @"^[A-Z][a-z]+(?:[s-][a-zA-Z]+)*$";
    tryToMatch = Regex.Match(txtTown.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid town (e.g., North London).", "Error");
        return false;
    }
    pattern = @"^([Gg][Ii][Rr] 0[Aa]{2})|((([A-Za-z][0-9]{1,2})|((([A-Za-z][A-Ha-hJ-Yj-y][0-9]{1,2})|((([A-Za-z][0-9][A-Za-z])|([A-Za-z][A-Ha-hJ-Yj-y][0-9][A-Za-z]?))\s?[0-9][A-Za-z]{2})$";
    tryToMatch = Regex.Match(txtPostcode.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid postcode (e.g., BH2 7JP).", "Error");
        return false;
    }
    return true;
}

internal void EditAccountAddress(string userID)
{
    altClearBtn = true;
    List<ClsUser> userList = new List<ClsUser>();
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    dbConnector.Connect();
    sqlStr = "SELECT (Surname & " + "'", "' + "& FirstName) as username FROM tblUser
WHERE UserID = " + userID;
    dr = dbConnector.DoSQL(sqlStr);
    dr.Read();
    userList.Add(new ClsUser { userid = Convert.ToInt32(userID), username =
dr[0].ToString() });
    cmbCustomerID.DisplayMember = "username";
    cmbCustomerID.ValueMember = "userid";
    cmbCustomerID.DataSource = userList;
    dbConnector.Close();
    LoadTable();
}

internal void SetUpNewCustomer()
{
    PopulateCombo();
    int highestUserID = FindHighestID();
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    string username = "";
    dbConnector.Connect();
    sqlStr = "SELECT (Surname & " + "'", "' + $"& FirstName) FROM tblUser WHERE UserID =
{highestUserID}";
    dr = dbConnector.DoSQL(sqlStr);
    while (dr.Read())
    {
        username = dr[0].ToString();
    }
    dbConnector.Close();
    cmbCustomerID.Text = username;
    cmbCustomerID.SelectedValue = highestUserID;
}

```

```

private int FindHighestID()
{
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    dbConnector.Connect();
    sqlStr = "SELECT MAX(UserID) FROM tblUser";
    dr = dbConnector.DoSQL(sqlStr);
    while (dr.Read())
    {
        return Convert.ToInt32(dr[0]);
    }
    dbConnector.Close();
    return 0;
}

private void lstAddresses_SelectedIndexChanged(object sender, EventArgs e)
{
    if (lstAddresses.SelectedItems.Count > 0)
    {
        ListViewItem selectedItem = lstAddresses.SelectedItems[0];
        selectedAddressID = selectedItem.SubItems[0].Text;

        clsDBConnector dbConnector = new clsDBConnector();
        OleDbDataReader dr;
        string sqlStr;
        dbConnector.Connect();
        sqlStr = "SELECT FirstLine, Town, Postcode FROM tblAddress WHERE AddressID = " +
selectedAddressID;
        dr = dbConnector.DoSQL(sqlStr);
        dr.Read();
        txtFirstLine.Text = dr[0].ToString();
        txtTown.Text = dr[1].ToString();
        txtPostcode.Text = dr[2].ToString();
        dbConnector.Close();
    }
}

private void btnDelete_Click(object sender, EventArgs e)
{
    if (selectedAddressID == "")
    {
        MessageBox.Show("You need to select an address to delete from the list.",
"Error");
    }
    else
    {
        clsDBConnector dbConnector = new clsDBConnector();
        DialogResult dialogResult = MessageBox.Show($"Are you sure that you want to
delete {GetFirstName()}'s address, which is: \n{GetAddress()}?", "Deleting Address",
MessageBoxButtons.YesNo);
        if (dialogResult == DialogResult.Yes)
        {
            string cmdStr = "DELETE FROM tblAddress " +
                $"WHERE AddressID = {selectedAddressID}";
            dbConnector.Connect();
            dbConnector.DoDML(cmdStr);
            dbConnector.Close();
            ClearFields();
            cmbCustomerID_SelectedIndexChanged(sender, e);
        }
    }
}

private string GetAddress()
{
    string addressToDelete = "";
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    dbConnector.Connect();
    sqlStr = "SELECT FirstLine, Town, Postcode FROM tblAddress WHERE AddressID = " +
selectedAddressID;
    dr = dbConnector.DoSQL(sqlStr);
    dr.Read();

```

```

        addressToDelete = dr[0].ToString() + ", " + dr[1].ToString() + ", " +
dr[2].ToString();
        dbConnector.Close();
        return addressToDelete;
    }

    private string GetFirstName()
    {
        string firstName = "";
        clsDBConnector dbConnector = new clsDBConnector();
        OleDbDataReader dr;
        string sqlStr;
        dbConnector.Connect();
        sqlStr = $"SELECT FirstName FROM tblUser WHERE UserID =
{cmbCustomerID.SelectedValue}";
        dr = dbConnector.DoSQL(sqlStr);
        dr.Read();
        firstName = dr[0].ToString();
        dbConnector.Close();
        return firstName;
    }

    private void btnClear_Click(object sender, EventArgs e)
    {
        ClearFields();
        if (!altClearBtn)
        {
            cmbCustomerID.Text = "Select";
            lstAddresses.Items.Clear();
        }
    }

    private void btnUpdate_Click(object sender, EventArgs e)
    {
        if (selectedAddressID == "")
        {
            "Error");
            MessageBox.Show("You need to select an address to update from the list.",
        }
        else if (CheckValid() == true)
        {
            clsDBConnector dbConnector = new clsDBConnector();
            string cmdStr = "UPDATE tblAddress " +
                $"SET FirstLine = '{txtFirstLine.Text}'," +
                $"Town = '{txtTown.Text}'," +
                $"Postcode = '{txtPostcode.Text}' " +
                $"WHERE (AddressID = {selectedAddressID})";
            dbConnector.Connect();
            dbConnector.DoDML(cmdStr);
            dbConnector.Close();
            cmbCustomerID_SelectedIndexChanged(sender, e);
        }
    }

    private void frmAddresses_FormClosed(object sender, FormClosedEventArgs e)
    {
        if (Application.OpenForms["frmMain"] as frmMain != null)
        {
            (Application.OpenForms["frmMain"] as frmMain).frmAddrOpen = false;
        }
        if (Application.OpenForms["frmAccount"] as frmAccount != null)
        {
            (Application.OpenForms["frmAccount"] as frmAccount).frmAddrOpen = false;
        }
    }

    private void cmbCustomerID_KeyPress(object sender, KeyPressEventArgs e)
    {
        e.Handled = true;
    }
}

```

5.4 Class Cart

frmCart

The screenshot shows a window titled "Reinforcements - Cart". Inside, there is a table titled "Cart Contents". The table has five columns: "Product", "Quantity", "Length", "Price (ex VAT)", and "Price (inc VAT)". There are two rows of items, each with a small icon of a rebar to its left. The first row shows "ReBar 12mm Mild Steel" with a quantity of 10, a custom length of 2.5m (2500mm), and prices of £86 (ex VAT) and £103.2 (inc VAT). The second row shows "ReBar 8mm Mild Steel" with a quantity of 1, a length of 1m (1000mm), and prices of £1.91 (ex VAT) and £2.29 (inc VAT). Below the table, a "Total:" row shows a total price of £87.91 (ex VAT) and £105.49 (inc VAT). At the bottom right, there is a "Checkout" button.

Product	Quantity	Length	Price (ex VAT)	Price (inc VAT)
ReBar 12mm Mild Steel	10	Custom length: 2.5m (2500mm)	£86	£103.2
ReBar 8mm Mild Steel	1	Length: 1m (1000mm)	£1.91	£2.29
Total:			£87.91	£105.49

Checkout

C# code

namespace Coursework

```
{
    public partial class frmCart : Form
    {
        public frmCart()
        {
            InitializeComponent();

            List<string> productNameLst = new List<string>();
            List<int> diamLst = new List<int>();
            List<string> materialLst = new List<string>();
            List<int> qtyLst = new List<int>();
            List<double> lengthLst = new List<double>();
            List<string> cartIDLst = new List<string>();
            List<int> stockLst = new List<int>();
            List<double> priceLst = new List<double>();
            List<int> productIDLst = new List<int>();

            Label lblTotalNoVat = new Label();
            Label lblTotalIncVat = new Label();

            Font font = new Font("Microsoft Sans Serif", 8.25f);
            Font font2 = new Font("Microsoft Sans Serif", 14.25f, FontStyle.Bold);

            bool emptyCart = false;
            string currentUserID;
            public bool frmCheckoutOpen = false;

            private frmCheckout frmCheckout = null;

            private void frmCart_FormClosed(object sender, FormClosedEventArgs e)
            {
                if (Application.OpenForms["frmMain"] as frmMain != null)
                {
                    (Application.OpenForms["frmMain"] as frmMain).frmCartOpen = false;
                }
            }

            internal void LoadDetails(string userID)
            {
                currentUserID = userID;
                clsDBConnector dbConnector = new clsDBConnector();
                OleDbDataReader dr;
                string sqlStr;
                dbConnector.Connect();
                sqlStr = $"SELECT ProductName, Quantity, DiameterInMM, Material, tblCart.LengthInM,
CartID, AmountInStock, PricePerMeter, tblCart.ProductID FROM tblCart, tblProduct " +
                    $"WHERE UserID = {userID} AND tblCart.ProductID = tblProduct.ProductID
ORDER BY Quantity DESC";
            }
        }
    }
}
```

```

dr = dbConnector.DoSQL(sqlStr);
int heightMultiplier = 1;
while (dr.Read())
{
    productNameLst.Add(dr[0].ToString());
    qtyLst.Add(Convert.ToInt32(dr[1]));
    diamLst.Add(Convert.ToInt32(dr[2]));
    materialLst.Add(dr[3].ToString());
    lengthLst.Add(Convert.ToDouble(dr[4]));
    cartIDLst.Add(dr[5].ToString());
    stockLst.Add(Convert.ToInt32(dr[6]));
    priceLst.Add(Convert.ToDouble(dr[7]));
    productIDLst.Add(Convert.ToInt32(dr[8]));
    heightMultiplier += 1;
}
if (heightMultiplier == 1)
{
    emptyCart = true;
}
this.Size = new Size(820, 150 + 60 * heightMultiplier);
dbConnector.Close();
LoadControls();
}

private void LoadControls()
{
    int i = 0;
    int x = 15;
    int y = 57;
    if (emptyCart)
    {
        x = 62;
        y = 100;
        Label lbl = new Label();
        lbl.Visible = true;
        lbl.ForeColor = SystemColors.ControlText;
        lbl.BackColor = SystemColors.Window;
        lbl.AutoSize = true;
        lbl.Size = new Size(64, 13);
        lbl.Location = new Point(x, y);
        lbl.Name = "lblEmptyCart";
        lbl.Text = "You have not added any products to the cart...";
        lbl.Font = new Font("Microsoft Sans Serif", 8.25f, FontStyle.Italic);
        y += 60;
        this.Controls.Add(lbl);
    }
    foreach (var material in materialLst)
    {
        PictureBox pic = new PictureBox();
        pic.Visible = true;
        if (material.ToLower() == "mild steel")
        {
            pic.Image = Image.FromFile("rebar_mild_steel.png");
        }
        else if (material.ToLower() == "stainless steel")
        {
            pic.Image = Image.FromFile("rebar_stainless_steel.png");
        }
        pic.SizeMode = PictureBoxSizeMode.StretchImage;
        pic.Size = new Size(40, 40);
        pic.Location = new Point(x, y);
        pic.Name = "picBox_" + i;
        this.Controls.Add(pic);

        PictureBox outline = new PictureBox();
        outline.Visible = true;
        outline.BackColor = Color.DarkSlateGray;
        outline.Size = new Size(42, 42);
        outline.Location = new Point(x - 1, y - 1);
        outline.Name = "outline_" + i;
        this.Controls.Add(outline);
        i++;
        y += 60;
    }
}

```

```

i = 0;
x += 49;
y = 57;
foreach (var product in productNameLst)
{
    Label lbl = new Label();
    lbl.Visible = true;
    lbl.ForeColor = SystemColors.ControlText;
    lbl.BackColor = SystemColors.Window;
    lbl.AutoSize = true;
    lbl.Size = new Size(64, 13);
    lbl.Location = new Point(x, y);
    lbl.Name = "lblProduct_" + i;
    lbl.Text = $"{product}\n{diamLst[i]}mm\n{materialLst[i]}";
    lbl.Font = font;
    i++;
    y += 60;
    this.Controls.Add(lbl);
}

i = 0;
x += (2 * 49);
y = 67;
foreach (var qty in qtyLst)
{
    NumericUpDown qtyPicker = new NumericUpDown();
    qtyPicker.Visible = true;
    qtyPicker.BorderStyle = BorderStyle.FixedSingle;
    qtyPicker.Font = font;
    qtyPicker.TextAlign = HorizontalAlignment.Center;
    qtyPicker.AutoSize = true;
    qtyPicker.Size = new Size(60, 15);
    qtyPicker.Location = new Point(x, y);
    qtyPicker.Maximum = Convert.ToInt32((stockLst[i] * 12) / lengthLst[i]);
    qtyPicker.Value = Convert.ToInt32(qty);
    qtyPicker.Minimum = 1;
    qtyPicker.Name = "qtyPicker_" + i;
    qtyPicker.Tag = i;
    qtyPicker.ValueChanged += qtyPicker_ValueChanged;
    this.Controls.Add(qtyPicker);
    i++;
    y += 60;
}

i = 0;
x += (49 + 60);
y = 69;
foreach (var length in lengthLst)
{
    Label lblLength = new Label();
    lblLength.Visible = true;
    lblLength.ForeColor = SystemColors.ControlText;
    lblLength.BackColor = SystemColors.Window;
    lblLength.AutoSize = true;
    lblLength.Size = new Size(64, 13);
    lblLength.Location = new Point(x, y);
    lblLength.Name = "lblLength_" + i;
    lblLength.Text = $"length: {length}m ({length * 1000}mm)";
    if (length == 1)
    {
        lblLength.Text = "L" + lblLength.Text;
    }
    else
    {
        lblLength.Text = "Custom L" + lblLength.Text;
    }
    lblLength.Font = font;
    lblLength.Tag = i;
    i++;
    y += 60;
    this.Controls.Add(lblLength);
}

```

```

i = 0;
x = 765;
y = 66;
foreach (var item in productNameLst)
{
    PictureBox trash = new PictureBox();
    trash.Visible = true;
    trash.Cursor = Cursors.Hand;
    trash.SizeMode = PictureBoxSizeMode.StretchImage;
    trash.Image = Image.FromFile("trash.png");
    trash.Size = new Size(15, 15);
    trash.Location = new Point(x, y);
    trash.Name = "picTrash_" + i;
    trash.Tag = i;
    trash.Click += PicTrash_Click;
    this.Controls.Add(trash);
    i++;
    y += 60;
}
if (!emptyCart)
{
    LoadPrices();
}
}

private void PicTrash_Click(object sender, EventArgs e)
{
    DialogResult dialogResult = MessageBox.Show($"Are you sure that you want to remove
that product from the cart?", "Removing From Cart", MessageBoxButtons.YesNo);
    if (dialogResult == DialogResult.Yes)
    {
        int i = Convert.ToInt32((sender as PictureBox).Tag);
        clsDBConnector dbConnector = new clsDBConnector();
        string cmdStr = $"DELETE FROM tblCart WHERE CartID = {cartIDLst[i]}";
        dbConnector.Connect();
        dbConnector.DoDML(cmdStr);
        dbConnector.Close();
        this.Close();
        frmCart frmCart = new frmCart();
        frmCart.Show();
        (Application.OpenForms["frmMain"] as frmMain).frmCartOpen = true;
        frmCart.LoadDetails(currentUserID);
    }
}

private void LoadPrices()
{
    int i = 0;
    int x = 500;
    int y = 69;
    foreach (var price in priceLst)
    {
        Label lblPriceNoVat = new Label();
        lblPriceNoVat.Visible = true;
        lblPriceNoVat.ForeColor = SystemColors.ControlText;
        lblPriceNoVat.BackColor = SystemColors.Window;
        lblPriceNoVat.AutoSize = true;
        lblPriceNoVat.Size = new Size(64, 13);
        lblPriceNoVat.Location = new Point(x, y);
        lblPriceNoVat.Name = "lblPriceNoVat_" + i;
        lblPriceNoVat.Text = $"£{Math.Round(price * lengthLst[i] * qtyLst[i], 2)}";
        lblPriceNoVat.Font = font;
        i++;
        y += 60;
        this.Controls.Add(lblPriceNoVat);
    }
    i = 0;
    x = 650;
    y = 69;
}

```

```

foreach (var price in priceLst)
{
    Label lblPriceIncVat = new Label();
    lblPriceIncVat.Visible = true;
    lblPriceIncVat.ForeColor = SystemColors.ControlText;
    lblPriceIncVat.BackColor = SystemColors.Window;
    lblPriceIncVat.AutoSize = true;
    lblPriceIncVat.Size = new Size(64, 13);
    lblPriceIncVat.Location = new Point(x, y);
    lblPriceIncVat.Name = "lblPriceIncVat_" + i;
    lblPriceIncVat.Text = $"{Math.Round(1.2 * (price * lengthLst[i] * qtyLst[i]),
2)}}";

    lblPriceIncVat.Font = font;
    i++;
    y += 60;
    this.Controls.Add(lblPriceIncVat);
}

Label lblTotal = new Label();
lblTotal.ForeColor = SystemColors.ControlText;
lblTotal.BackColor = SystemColors.Window;
lblTotal.AutoSize = true;
lblTotal.Size = new Size(64, 13);
lblTotal.Name = "lblTotal";
lblTotal.Font = font2;
lblTotal.Location = new Point(270, y - 7);
lblTotal.Visible = true;
lblTotal.Text = $"Total:";
this.Controls.Add(lblTotal);

lblTotalNoVat.ForeColor = SystemColors.ControlText;
lblTotalNoVat.BackColor = SystemColors.Window;
lblTotalNoVat.AutoSize = true;
lblTotalNoVat.Size = new Size(64, 13);
lblTotalNoVat.Name = "lblTotalNoVat";
lblTotalNoVat.Font = font;
lblTotalNoVat.Location = new Point(500, y);
lblTotalNoVat.Visible = true;
lblTotalNoVat.Text = "";
this.Controls.Add(lblTotalNoVat);

lblTotalIncVat.ForeColor = SystemColors.ControlText;
lblTotalIncVat.BackColor = SystemColors.Window;
lblTotalIncVat.AutoSize = true;
lblTotalIncVat.Size = new Size(64, 13);
lblTotalIncVat.Name = "lblTotalIncVat";
lblTotalIncVat.Font = new Font("Microsoft Sans Serif", 8.25f, FontStyle.Underline);
lblTotalIncVat.Location = new Point(650, y);
lblTotalIncVat.Visible = true;
lblTotalIncVat.Text = "";
this.Controls.Add(lblTotalIncVat);

UploadTotals();
y += 50;

Button btnCheckout = new Button();
btnCheckout.Size = new Size(75, 21);
btnCheckout.Location = new Point(640, y);
btnCheckout.Visible = true;
btnCheckout.Font = new Font("Microsoft Sans Serif", 8.25f, FontStyle.Bold);
btnCheckout.Text = "Checkout";
btnCheckout.Name = "btnCheckout";
btnCheckout.UseVisualStyleBackColor = true;
btnCheckout.Click += btnCheckout_Click;
this.Controls.Add(btnCheckout);
}

```



```

private void btnCheckout_Click(object sender, EventArgs e)
{
    if (!frmCheckoutOpen)
    {
        frmCheckoutOpen = true;
        frmCheckout = new frmCheckout();
        frmCheckout.Show();
        frmCheckout.LoadDetails(currentUserID, total);
    }
    else
    {
        frmCheckout.BringToFront();
        frmCheckout.WindowState = FormWindowState.Normal;
    }
}

double total = 0;

private void UploadTotals()
{
    int i = 0;
    foreach (var singlePrice in priceLst)
    {
        total += singlePrice * lengthLst[i] * qtyLst[i];
        i++;
    }
    lblTotalNoVat.Text = $"£{Math.Round(total, 2)}";
    lblTotalIncVat.Text = $"£{Math.Round(1.2 * total, 2)}";
}

private void qtyPicker_ValueChanged(object sender, EventArgs e)
{
    clsDBConnector dbConnector = new clsDBConnector();
    int i = Convert.ToInt32((sender as NumericUpDown).Tag);
    string cmdStr = $"UPDATE tblCart SET Quantity = '{(sender as NumericUpDown).Value}'
WHERE CartID = {cartIDLst[i]}";
    dbConnector.Connect();
    dbConnector.DoDML(cmdStr);
    dbConnector.Close();
    qtyLst[i] = (int)(sender as NumericUpDown).Value;

    foreach (Control c in this.Controls)
    {
        if (c is Label lblPriceNoVat && lblPriceNoVat.Name == "lblPriceNoVat_" + i)
        {
            lblPriceNoVat.Text = $"£{Math.Round(priceLst[i] * lengthLst[i] * qtyLst[i],
2)}";
            break;
        }
    }

    foreach (Control c in this.Controls)
    {
        if (c is Label lblPriceIncVat && lblPriceIncVat.Name == "lblPriceIncVat_" + i)
        {
            lblPriceIncVat.Text = $"£{Math.Round(1.2 * (priceLst[i] * lengthLst[i] *
qtyLst[i]), 2)}";
            break;
        }
    }
    UploadTotals();
}

private void frmCart_Load(object sender, EventArgs e)
{
    Label lblTotalNoVat = new Label();
    lblTotalNoVat.Visible = false;
    lblTotalNoVat.ForeColor = SystemColors.ControlText;
    lblTotalNoVat.BackColor = SystemColors.Window;
    lblTotalNoVat.AutoSize = true;
    lblTotalNoVat.Size = new Size(64, 13);
    lblTotalNoVat.Name = "lblTotalNoVat";
    lblTotalNoVat.Font = font;
    this.Controls.Add(lblTotalNoVat);
}

```

```

        Label lblTotalExVat = new Label();
        lblTotalExVat.Visible = false;
        lblTotalExVat.ForeColor = SystemColors.ControlText;
        lblTotalExVat.BackColor = SystemColors.Window;
        lblTotalExVat.AutoSize = true;
        lblTotalExVat.Size = new Size(64, 13);
        lblTotalExVat.Name = "lblTotalExVat";
        lblTotalExVat.Font = font;
        this.Controls.Add(lblTotalExVat);
    }

    internal void OrderCompleted(string addressID)
    {
        clsDBConnector dbConnector = new clsDBConnector();
        OleDbDataReader dr;
        string cmdStr = $"DELETE FROM tblCart WHERE UserID = {currentUserID}";
        dbConnector.Connect();
        dbConnector.DoDML(cmdStr);

        cmdStr = $"INSERT INTO tblOrder (UserID, DateOfOrder, TotalPaid, AddressID) " +
            $"VALUES ('{currentUserID}', '{DateTime.Now.ToString("dd/MM/yyyy")}', " +
            '{Math.Round(1.2 * total, 2)}', '{addressID}')';
        dbConnector.DoDML(cmdStr);

        string sqlStr = $"SELECT MAX(OrderID) FROM tblOrder";
        dr = dbConnector.DoSQL(sqlStr);
        dr.Read();
        string orderID = dr[0].ToString();

        int i = 0;
        foreach (var productID in productIDLst)
        {
            cmdStr = $"INSERT INTO tblProductOrder (OrderID, ProductID, Quantity, LengthInM) " +
                $"VALUES ('{orderID}', '{productID}', '{qtyLst[i]}', " +
                '{lengthLst[i]}')';
            dbConnector.DoDML(cmdStr);
            i++;
        }
        i = 0;
        int piecesPerBar = (int)(12 / lengthLst[i]);
        if (piecesPerBar == 0)
        {
            piecesPerBar = 1;
        }
        foreach (var productID in productIDLst)
        {
            cmdStr = $"UPDATE tblProduct SET AmountInStock = AmountInStock - " +
                "({Math.Ceiling((double)qtyLst[i] / piecesPerBar)}) WHERE ProductID = {productID}";
            dbConnector.DoDML(cmdStr);
            i++;
        }
        dbConnector.Close();
    }
}

```

5.5 Class Checkout

frmCheckout

Reinforcements - Checkout

— □ ×

Shipping

Delivery address

2 Bohemia Road, Boumemouth, BH11 9FT ▾

Payment

Card holder

Card number

Expiration date

/

CVC

Summary

Subtotal £87.91

Shipping FREE

VAT £17.58

Total £105.49

Place order

We Accept




C# code

namespace Coursework

```
{
    public partial class frmCheckout : Form
    {
        public frmCheckout()
        {
            InitializeComponent();
        }
        class CLsAddress
        {
            public int addressID { get; set; }
            public string address { get; set; }
        }

        string cardName;
        string cardNumber;
        string expirationMM;
        string expirationYY;
        string CVC;
        double total;

        internal void LoadDetails(string currentUserID, double subtotal)
        {
            List<CLsAddress> addressList = new List<CLsAddress>();
            clsDBConnector dbConnector = new clsDBConnector();
            OleDbDataReader dr;
            string sqlStr;
            dbConnector.Connect();
            sqlStr = "SELECT AddressID, FirstLine, Town, Postcode" +
                " FROM tblAddress" +
                " WHERE UserID = " + currentUserID;
            dr = dbConnector.DoSQL(sqlStr);
            while (dr.Read())
            {
                addressList.Add(new CLsAddress { addressID = Convert.ToInt32(dr[0]), address =
$"{dr[1]}, {dr[2]}, {dr[3]}" });
            }
            dbConnector.Close();
            cmbDeliveryAddr.DisplayMember = "address";
        }
    }
}
```

```

        cmbDeliveryAddr.ValueMember = "addressID";
        cmbDeliveryAddr.DataSource = addressList;

        lblSubtotalValue.Text = $"£{Math.Round(subtotal, 2)}";
        lblVATValue.Text = $"£{Math.Round(1.2 * subtotal - subtotal, 2)}";
        total = Math.Round(1.2 * subtotal, 2);
        lblTotalValue.Text = $"£{total}";
    }

    private bool CheckValid()
    {
        if (txtCardName.Text == "" || txtCardNumber.Text == "" || txtMM.Text == "" ||
            txtYY.Text == "" || txtCVC.Text == "")
        {
            MessageBox.Show("You have not entered anything for one or many fields.",
                "Error");
            return false;
        }
        cardName = txtCardName.Text;
        string pattern = @"^[A-Za-z]+(?: [A-Za-z]+)*$";
        Match tryToMatch = Regex.Match(txtCardName.Text, pattern);
        if (!tryToMatch.Success || txtCardName.Text.Length < 2)
        {
            MessageBox.Show("Invalid cardholder name. It must contain only letters and
                spaces.", "Error");
            return false;
        }
        cardNumber = txtCardNumber.Text.Replace(" ", "");
        if (!long.TryParse(cardNumber, out _) || cardNumber.Length != 16)
        {
            MessageBox.Show("Invalid card number. It must be exactly 16 digits.", "Error");
            return false;
        }
        expirationMM = txtMM.Text.Replace(" ", "");
        expirationYY = txtYY.Text.Replace(" ", "");
        if (!long.TryParse(expirationMM, out _) || expirationMM.Length != 2 ||
            !long.TryParse(expirationYY, out _) || expirationYY.Length != 2 || expirationMM == "00" ||
            Convert.ToInt32(expirationMM) > 12)
        {
            MessageBox.Show("Invalid expiration date. Both fields must contain exactly 2
                digits.", "Error");
            return false;
        }
        CVC = txtCVC.Text.Replace(" ", "");
        if (!long.TryParse(CVC, out _) || (CVC.Length != 3 && CVC.Length != 4))
        {
            MessageBox.Show("Invalid card verification code (CVC). It must contain 3 or 4
                digits.", "Error");
            return false;
        }
        return true;
    }

    Timer timer = new Timer();

    private void btnPlaceOrder_Click(object sender, EventArgs e)
    {
        if (CheckValid())
        {
            if (TakeMoney(total, cardName, cardNumber, expirationMM, expirationYY, CVC))
            {
                lblAnimation.Text = ".";
                lblAnimation.Visible = true;
                timer.Interval = 1000;
                timer.Tick += Timer_Tick;
                timer.Start();
            }
            else
            {
                MessageBox.Show("There has been a problem, try using different payment
                    method.", "Payment Failure");
            }
        }
    }
}

```

```

        private bool TakeMoney(double total, string cardName, string cardNumber, string
expirationMM, string expirationYY, string cvc)
        {
            string today = DateTime.Now.ToString("MM/yyyy");
            string currentMonth = today[0] + "" + today[1];
            string currentYear = today[5] + "" + today[6];
            if (Convert.ToInt32(expirationYY) < Convert.ToInt32(currentYear))
            {
                return false;
            }
            if (Convert.ToInt32(expirationYY) == Convert.ToInt32(currentYear) &&
Convert.ToInt32(expirationMM) < Convert.ToInt32(currentMonth))
            {
                return false;
            }
            if (true)
            {
                //other validations
            }
            return true;
        }

        int count = 0;

        private void Timer_Tick(object sender, EventArgs e)
        {
            if (lblAnimation.Text != "...")
            {
                lblAnimation.Text += ".";
            }
            else
            {
                lblAnimation.Text = ".";
            }
            if (count == 5)
            {
                timer.Stop();
                lblAnimation.Visible = false;
                MessageBox.Show("Order completed. Thank you!");
                (Application.OpenForms["frmCart"] as
frmCart).OrderCompleted(cmbDeliveryAddr.Selected.Value.ToString());
                this.Close();
                Form frmCart = Application.OpenForms["frmCart"];
                if (frmCart != null)
                {
                    frmCart.Close();
                    (Application.OpenForms["frmMain"] as frmMain).frmCartOpen = false;
                }
                (Application.OpenForms["frmMain"] as frmMain).DisplayData(false);
            }
            count++;
        }

        private void frmCheckout_FormClosed(object sender, FormClosedEventArgs e)
        {
            if (Application.OpenForms["frmCart"] as frmCart != null)
            {
                (Application.OpenForms["frmCart"] as frmCart).frmCheckoutOpen = false;
            }
        }

        private void cmbDeliveryAddr_KeyPress(object sender, KeyPressEventArgs e)
        {
            e.Handled = true;
        }
    }
}

```

5.6 Class Edit Details

frmEditDetails

Edit Details

First name: Peter

Surname: Ivanov

Email: whynotnoll1e@gmail.com

Phone number: 07927369011

Company name:

Password:

Confirm password:

Keep the password field clear if you do not want it changed

C# code

namespace Coursework

```
{
    public partial class frmEditDetails : Form
    {
        public frmEditDetails()
        {
            InitializeComponent();
        }

        private void frmEditDetails_Load(object sender, EventArgs e)
        {
        }

        private string GetHashSHA256(string plainText)
        {
            string hashText = "";
            Encoding enc = Encoding.UTF8;
            SHA256Managed hash = new SHA256Managed();
            byte[] result = hash.ComputeHash(enc.GetBytes(plainText));
            foreach (Byte item in result)
            {
                hashText = hashText + item.ToString("X");
            }
            return hashText;
        }

        string userID;

        internal void LoadDetails(string v, string firstName, string surname, string email,
            string phoneNumber, string companyName)
        {
            userID = v;
            txtFirstName.Text = firstName;
            txtSurname.Text = surname;
            txtEmail.Text = email;
            txtPhoneNumber.Text = phoneNumber;
            txtCompanyName.Text = companyName;
        }
    }
}
```

```

private void picViewPassword_MouseUp(object sender, MouseEventArgs e)
{
    txtPassword.PasswordChar = '*';
    picViewPassword.Image = Image.FromFile("view_password.png");
}

private void picViewPassword_MouseDown(object sender, MouseEventArgs e)
{
    txtPassword.PasswordChar = Convert.ToChar(0);
    picViewPassword.Image = Image.FromFile("hide_password.png");
}

private void btnUpdate_Click(object sender, EventArgs e)
{
    if (CheckValid() == true)
    {
        DialogResult dialogResult = MessageBox.Show($"Are you sure that you want to
update your details?", "Updating Details", MessageBoxButtons.YesNo);
        if (dialogResult == DialogResult.Yes)
        {
            clsDBConnector dbConnector = new clsDBConnector();
            string cmdStr = "UPDATE tblUser " +
                $"SET FirstName = '{txtFirstName.Text}', " +
                $"Surname = '{txtSurname.Text}', " +
                $"Email = '{txtEmail.Text}', " +
                $"PhoneNumber = '{txtPhoneNumber.Text}', " +
                $"CompanyName = '{txtCompanyName.Text}'";

            if (txtPassword.Text != "")
            {
                cmdStr = cmdStr + $", [Password] = '{GetHashSHA256(txtPassword.Text)}'";
            }

            cmdStr = cmdStr + $"WHERE (UserID = {userID})";
            dbConnector.Connect();
            dbConnector.DoDML(cmdStr);
            dbConnector.Close();
            (Application.OpenForms["frmMain"] as frmMain).DisplayData(false);
            (Application.OpenForms["frmAccount"] as frmAccount).frmAccount_Load(sender,
e);
        }
    }
}

private bool CheckValid()
{
    if (txtFirstName.Text == "" || txtSurname.Text == "" || txtEmail.Text == "" ||
txtPhoneNumber.Text == "")
    {
        MessageBox.Show("You have not entered anything for one or many fields.",
"Error");
        return false;
    }
    string pattern = @"[A-Z][a-z]+";
    Match tryToMatch = Regex.Match(txtFirstName.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid first name (e.g., Charlie).", "Error");
        return false;
    }
    pattern = @"[A-Z][a-z]+";
    tryToMatch = Regex.Match(txtSurname.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid surname (e.g., Black).", "Error");
        return false;
    }
    pattern = @"(?:[a-z0-9!#$%&'*/+=?^_`{|}~]+(?:\. [a-z0-9!#$%&'*/+=?^_`{|}~
+)*|""(?:[\\x01-\\x08\\x0c\\x0e-\\x1f\\x21\\x23-\\x5b\\x5d-\\x7f]|\\[\\x01-\\x09\\x0b\\x0c\\x0e-
\\x7f])""))*"@;
    tryToMatch = Regex.Match(txtEmail.Text, pattern);
}

```


5.7 Class Manage Users

frmManageUsers

C# code

```
namespace Coursework
{
    public partial class frmManageUsers : Form
    {
        public frmManageUsers()
        {
            InitializeComponent();
        }

        class CLsUser
        {
            public int userid { get; set; }
            public string username { get; set; }
        }

        private void btnAddCustomer_Click(object sender, EventArgs e)
        {
            if (CheckValid() == true)
            {
                clsDBConnector dbConnector = new clsDBConnector();
                string cmdStr = $"INSERT INTO tblUser (FirstName, Surname, Email, PhoneNumber,
CompanyName, Manager) " +
                    $"VALUES ('{txtFirstName.Text}', '{txtSurname.Text}',
'{txtEmail.Text}', '{txtPhoneNumber.Text}', '{txtCompanyName.Text}', {chkManager.Checked})";
                dbConnector.Connect();
                dbConnector.DoDML(cmdStr);
                dbConnector.Close();
                (Application.OpenForms["frmMain"] as frmMain).DisplayData(false);
                DialogResult dialogResult = MessageBox.Show($"Do you want to associate an
address with {txtFirstName.Text}?", "Adding Address", MessageBoxButtons.YesNo);
                if (dialogResult == DialogResult.Yes)
                {
                    frmAddresses frmAddresses = new frmAddresses();
                    frmAddresses.Show();
                    frmAddresses.SetupNewCustomer();
                }
                frmManageCustomers_Load(sender, e);
            }
        }
    }
}
```

```

private bool CheckValid()
{
    if (txtFirstName.Text == "" || txtSurname.Text == "" || txtEmail.Text == "" ||
txtPhoneNumber.Text == "")
    {
        MessageBox.Show("You have not entered anything for one or many fields.",
"Error");
        return false;
    }
    string pattern = @"[A-Z][a-z]+";
    Match tryToMatch = Regex.Match(txtFirstName.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid first name (e.g., Charlie).", "Error");
        return false;
    }
    pattern = @"[A-Z][a-z]+";
    tryToMatch = Regex.Match(txtSurname.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid surname (e.g., Black).", "Error");
        return false;
    }
    pattern = @"(?:[a-z0-9!#$%&'*/+=?^_`{|}~]+(?:\. [a-z0-9!#$%&'*/+=?^_`{|}~
+)*|" + "(?:[x01-x08\x0b\x0c\x0e-x1f\x21\x23-x5b\x5d-x7f]|\\[x01-x09\x0b\x0c\x0e-
\x7f])*" + ")@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-
9])?|\\(?: (?: (2(5[0-5]| [0-4][0-9])|1[0-9][0-9])| [1-9]?[0-9])\\.){3} (?: (2(5[0-5]| [0-4][0-9])|1[0-
9][0-9])| [1-9]?[0-9])| [a-z0-9-]*[a-z0-9]:(?: [x01-x08\x0b\x0c\x0e-x1f\x21-x5a\x53-
\x7f]|\\[x01-x09\x0b\x0c\x0e-x7f])+)" + ")";
    tryToMatch = Regex.Match(txtEmail.Text, pattern);
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid email address (e.g., charli3black999@gmail.com).",
"Error");
        return false;
    }
    pattern = @"^(?:(?:\((?:(?:0(?:0|11))\)?[\s-]?(?|\+)(44)\)?[\s-]?(?:\((?0)\)?[\s-
]?)?)|(?:\((?0)\)?(?:\d{5}\)?[\s-]?\d{4,5})|(?:\d{4}\)?[\s-]?(?:\d{5}|\d{3}[\s-
]?\d{3}))|(?:\d{3}\)?[\s-]?\d{3}[\s-]?\d{3,4})|(?:\d{2}\)?[\s-]?\d{4}[\s-]?\d{4}))(?:[\s-
]?(?:x|ext)\.?|#)\d{3,4})?$";
    if (!tryToMatch.Success)
    {
        MessageBox.Show("Invalid phone number. (e.g., 07635483912)", "Error");
        return false;
    }
}

if (txtEmail.Text != currentEmail)
{
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    dbConnector.Connect();
    sqlStr = $"SELECT UserID FROM tblUser WHERE Email = '{txtEmail.Text}'";
    dr = dbConnector.DoSQL(sqlStr);
    int i = 0;
    while (dr.Read())
    {
        if (dr[i] != cmbCustomerID.SelectedValue)
        {
            MessageBox.Show("The email address you have entered is already used.",
"Error");
            return false;
        }
        i++;
    }
}
return true;
}

private void frmManageCustomers_Load(object sender, EventArgs e)
{
    ClearFields();
    cmbCustomerID.Text = "Select";
}

```

```

private void ClearFields()
{
    string emptyStr = "";
    txtFirstName.Text = emptyStr;
    txtSurname.Text = emptyStr;
    txtEmail.Text = emptyStr;
    txtPhoneNumber.Text = emptyStr;
    txtCompanyName.Text = emptyStr;
    chkManager.Checked = false;
}

private void PopulateCombo()
{
    List<ClsUser> userList = new List<ClsUser>();
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    dbConnector.Connect();
    sqlStr = "SELECT UserID, (Surname & " + "'", "'" + "& FirstName) as username FROM
tblUser ORDER BY UserID DESC";
    dr = dbConnector.DoSQL(sqlStr);
    while (dr.Read())
    {
        userList.Add(new ClsUser { userid = Convert.ToInt32(dr[0]), username =
dr[1].ToString() });
    }
    cmbCustomerID.DisplayMember = "username";
    cmbCustomerID.ValueMember = "userid";
    cmbCustomerID.DataSource = userList;
    dbConnector.Close();
}

bool cancelChkEvent = false;
string currentFirstName = "user";
string currentEmail = "email@gmail.com";

private void cmbCustomerID_SelectedIndexChanged(object sender, EventArgs e)
{
    if (cmbCustomerID.SelectedValue != null)
    {
        clsDBConnector dbConnector = new clsDBConnector();
        OleDbDataReader dr;
        string sqlStr;
        dbConnector.Connect();
        sqlStr = "SELECT UserID, FirstName, Surname, Email, PhoneNumber, CompanyName,
Manager" +
                " FROM tblUser" +
                " WHERE UserID = " + cmbCustomerID.SelectedValue;
        dr = dbConnector.DoSQL(sqlStr);
        while (dr.Read())
        {
            txtFirstName.Text = dr[1].ToString();
            currentFirstName = dr[1].ToString();
            txtSurname.Text = dr[2].ToString();
            txtEmail.Text = dr[3].ToString();
            currentEmail = dr[3].ToString();
            txtPhoneNumber.Text = dr[4].ToString();
            txtCompanyName.Text = dr[5].ToString();
            cancelChkEvent = true;
            if (dr[6].ToString() == "True")
            {
                chkManager.Checked = true;
            }
            else { chkManager.Checked = false; }
        }
        dbConnector.Close();
        cancelChkEvent = false;
    }
}

```

```

private void btnUpdate_Click(object sender, EventArgs e)
{
    if (cmbCustomerID.Text != "Select")
    {
        if (CheckValid() == true)
        {
            DialogResult dialogResult = MessageBox.Show($"Are you sure that you want to
update {currentFirstName}'s details?", "Updating Details", MessageBoxButtons.YesNo);
            if (dialogResult == DialogResult.Yes)
            {
                clsDBConnector dbConnector = new clsDBConnector();
                string cmdStr = "UPDATE tblUser " +
                    $"SET FirstName = '{txtFirstName.Text}'," +
                    $"Surname = '{txtSurname.Text}'," +
                    $"Email = '{txtEmail.Text}'," +
                    $"PhoneNumber = '{txtPhoneNumber.Text}'," +
                    $"CompanyName = '{txtCompanyName.Text}'," +
                    $"Manager = {chkManager.Checked} " +
                    $"WHERE (UserID = {cmbCustomerID.SelectedValue})";
                dbConnector.Connect();
                dbConnector.DoDML(cmdStr);
                dbConnector.Close();
                (Application.OpenForms["frmMain"] as frmMain).DisplayData(false);
                frmManageCustomers_Load(sender, e);
            }
        }
        else
        {
            MessageBox.Show($"You have not selected a user to update.", "Error");
        }
    }
}

private void btnDelete_Click(object sender, EventArgs e)
{
    if (cmbCustomerID.Text == "Select")
    {
        MessageBox.Show($"You have not selected a user to delete.", "Error");
    }
    else
    {
        clsDBConnector dbConnector = new clsDBConnector();
        DialogResult dialogResult = MessageBox.Show($"Are you sure that you want to
delete {txtFirstName.Text}?", "Deleting User", MessageBoxButtons.YesNo);
        if (dialogResult == DialogResult.Yes)
        {
            string cmdStr = "DELETE FROM tblUser " +
                $"WHERE UserID = {cmbCustomerID.SelectedValue}";
            dbConnector.Connect();
            dbConnector.DoDML(cmdStr);
            dbConnector.Close();
            (Application.OpenForms["frmMain"] as frmMain).DisplayData(false);
            frmManageCustomers_Load(sender, e);
        }
    }
}

private void cmbCustomerID_MouseClick(object sender, MouseEventArgs e)
{
    if (cmbCustomerID.Text == "Select")
    {
        PopulateCombo();
    }
}

private void btnClear_Click(object sender, EventArgs e)
{
    frmManageCustomers_Load(sender, e);
}

```

```

private void chkManager_CheckedChanged(object sender, EventArgs e)
{
    if (chkManager.Checked && !cancelChkEvent)
    {
        string username = "the user";
        if (txtFirstName.Text != "")
        {
            username = txtFirstName.Text;
        }
        DialogResult dialogResult = MessageBox.Show($"Are you sure that you want to set
{username} as a manager?\nYou need to either add or update for the change to take place.",
"Adding Manager", MessageBoxButtons.YesNo);
        if (dialogResult == DialogResult.No)
        {
            chkManager.Checked = false;
        }
    }
}

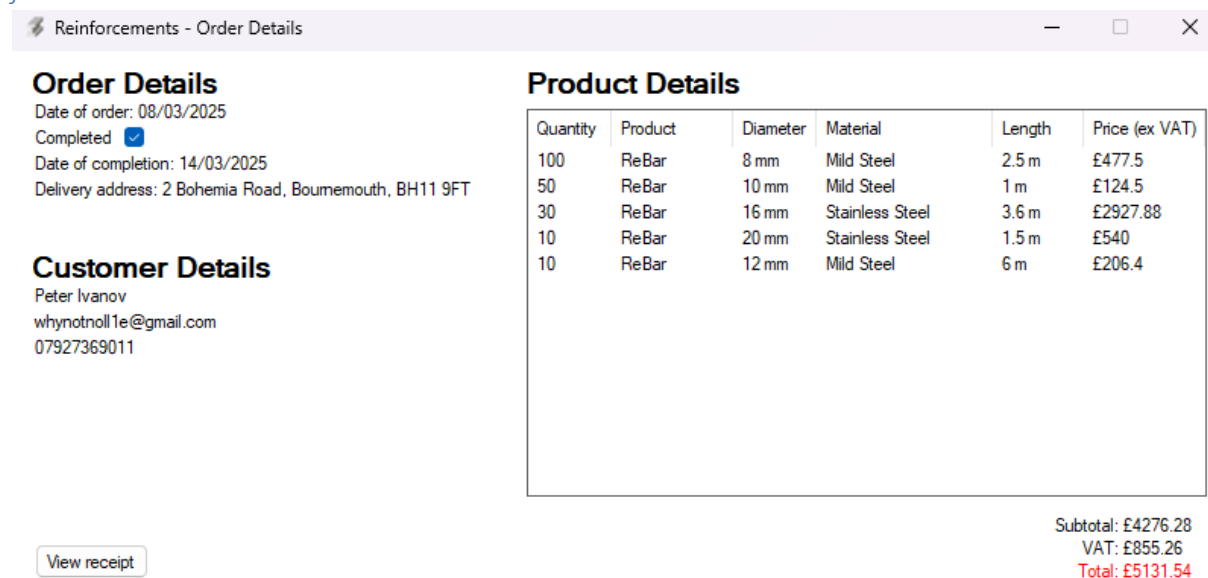
private void frmManageUsers_FormClosed(object sender, FormClosedEventArgs e)
{
    if (Application.OpenForms["frmMain"] as frmMain != null)
    {
        (Application.OpenForms["frmMain"] as frmMain).frmMUOpen = false;
    }
}

private void cmbCustomerID_KeyPress(object sender, KeyPressEventArgs e)
{
    e.Handled = true;
}
}
}

```

5.8 Class Order Details

frmOrderDetails



Order Details

Date of order: 08/03/2025

Completed ☒

Date of completion: 14/03/2025

Delivery address: 2 Bohemia Road, Boumemouth, BH11 9FT

Customer Details

Peter Ivanov

whynotnoll1e@gmail.com

07927369011

Product Details

Quantity	Product	Diameter	Material	Length	Price (ex VAT)
100	ReBar	8 mm	Mild Steel	2.5 m	£477.5
50	ReBar	10 mm	Mild Steel	1 m	£124.5
30	ReBar	16 mm	Stainless Steel	3.6 m	£2927.88
10	ReBar	20 mm	Stainless Steel	1.5 m	£540
10	ReBar	12 mm	Mild Steel	6 m	£206.4

View receipt

Subtotal: £4276.28

VAT: £855.26

Total: £5131.54

C# code

```

namespace Coursework
{
    public partial class frmOrderDetails : Form
    {
        public frmOrderDetails()
        {
            InitializeComponent();

            Random rnd = new Random();
            bool chkCompletedEvent = false;
            int orderID;
            double subtotal;
            double total;
            double VAT;
            string dateOfOrder;
            List<string> productsList = new List<string>();
            List<string> qtyList = new List<string>();
            List<string> priceList = new List<string>();

            internal void LoadDetails(string selectedOrderID)
            {
                clsDBConnector dbConnector = new clsDBConnector();
                OleDbDataReader dr;
                string sqlStr;
                dbConnector.Connect();
                sqlStr = "SELECT tblOrder.UserID, tblOrder.DateOfOrder, tblOrder.TotalPaid,
tblOrder.Completed, tblOrder.DateOfCompletion, (tblAddress.FirstLine & ', ' & tblAddress.Town &
', ' & tblAddress.Postcode) AS address " +
                "FROM (tblOrder INNER JOIN tblAddress ON tblOrder.AddressID =
tblAddress.AddressID) " +
                $"WHERE tblOrder.OrderID = {selectedOrderID}";
                dr = dbConnector.DoSQL(sqlStr);
                dr.Read();
                orderID = Convert.ToInt32(selectedOrderID);
                int userID = Convert.ToInt32(dr[0]);
                dateOfOrder = (Application.OpenForms["frmMain"] as
frmMain).GetDateOnly(dr[1]).ToString();
                lblDateOfOrder.Text += dateOfOrder;
                subtotal = Convert.ToDouble(dr[2]) / 1.2;
                total = Convert.ToDouble(dr[2]);
                VAT = total - subtotal;
                lblTotalPaidNoVAT.Text += "£" + Math.Round(subtotal, 2).ToString();
                lblVAT.Text += "£" + Math.Round(VAT, 2).ToString();
                lblTotalPaid.Text += "£" + Math.Round(total, 2).ToString();
                if (dr[3].ToString() == "True")
                {
                    chkCompleted.Checked = true;
                }
                lblDateOfCompletion.Text += (Application.OpenForms["frmMain"] as
frmMain).GetDateOnly(dr[4]).ToString();
                lblAddress.Text += dr[5].ToString();

                sqlStr = "SELECT Quantity, ProductName, DiameterInMM, Material, PricePerMeter,
LengthInM FROM tblProductOrder, tblProduct " +
                $"WHERE OrderID = {selectedOrderID} AND tblProductOrder.ProductID =
tblProduct.ProductID ORDER BY Quantity DESC";
                dr = dbConnector.DoSQL(sqlStr);
                lstProducts.Items.Clear();
                while (dr.Read())
                {
                    lstProducts.Items.Add(dr[0].ToString());
                    lstProducts.Items[lstProducts.Items.Count - 1].SubItems.Add((dr[1]).ToString());
                    lstProducts.Items[lstProducts.Items.Count - 1].SubItems.Add(dr[2].ToString() + "
mm");

                    string material = dr[3].ToString();
                    string materialShortVer = "";
                    lstProducts.Items[lstProducts.Items.Count - 1].SubItems.Add(material);

```

```

        for (int i = 0; i < material.Length; i++)
        {
            string original = material[i].ToString();
            string upper = material[i].ToString().ToUpper();
            if (original == upper && original != " ")
            {
                materialShortVer += original;
            }
        }
        double pricePP = Convert.ToDouble(dr[4]) * Convert.ToDouble(dr[0]) *
Convert.ToDouble(dr[5]);
        lstProducts.Items[lstProducts.Items.Count - 1].SubItems.Add(dr[5].ToString() + "
m");
        lstProducts.Items[lstProducts.Items.Count - 1].SubItems.Add("£" +
pricePP.ToString());

productsList.Add($"{dr[1].ToString().ToUpper()}{materialShortVer.ToLower()}{dr[2].ToString()}m
m{dr[5].ToString()}m");
        qtyList.Add(dr[0].ToString());
        priceList.Add($"£{pricePP.ToString()}");
    }

    sqlStr = $"SELECT FirstName, Surname, Email, PhoneNumber, CompanyName FROM tblUser
WHERE UserID = {userID}";
    dr = dbConnector.DoSQL(sqlStr);
    dr.Read();
    lblName.Text = dr[0].ToString() + " " + dr[1].ToString();
    lblEmail.Text = dr[2].ToString();
    lblPhoneNumber.Text = dr[3].ToString();
    lblCompanyName.Text = dr[4].ToString();
    dbConnector.Close();
    chkCompletedEvent = true;
}

private void chkCompleted_CheckedChanged(object sender, EventArgs e)
{
    if (chkCompletedEvent)
    {
        if (chkCompleted.Checked)
        {
            DialogResult dialogResult = MessageBox.Show($"Mark order #{orderId} as
complete?", "Completing Order", MessageBoxButtons.YesNo);
            if (dialogResult == DialogResult.Yes)
            {
                UpdateCompleteness(true);
            }
            else
            {
                chkCompletedEvent = false;
                chkCompleted.Checked = false;
            }
        }
        else
        {
            DialogResult dialogResult = MessageBox.Show($"Mark order #{orderId} as
uncomplete?", "Completing Order", MessageBoxButtons.YesNo);
            if (dialogResult == DialogResult.Yes)
            {
                UpdateCompleteness(false);
            }
            else
            {
                chkCompletedEvent = false;
                chkCompleted.Checked = true;
            }
        }
    }
    else
    {
        chkCompletedEvent = true;
    }
}

```

```

private void UpdateCompleteness(bool completed)
{
    clsDBConnector dbConnector = new clsDBConnector();
    string cmdStr = $"UPDATE tblOrder SET Completed = {completed}, ";
    if (completed)
    {
        string today = DateTime.Now.ToString("dd/MM/yyyy");
        cmdStr += $"DateOfCompletion = '{today}'";
        lblDateOfCompletion.Text = "Date of completion: " + today;
    }
    else
    {
        cmdStr += $"DateOfCompletion = null";
        lblDateOfCompletion.Text = "Date of completion: X";
    }
    cmdStr += $" WHERE OrderID = {orderId}";
    dbConnector.Connect();
    dbConnector.DoDML(cmdStr);
    dbConnector.Close();
    (Application.OpenForms["frmMain"] as frmMain).DisplayData(false);
}

private void btnReceipt_Click(object sender, EventArgs e)
{
    int receiptLength = 450;
    foreach (var item in productsList)
    {
        receiptLength += 15;
    }
    PrintDocument receipt;
    receipt = new PrintDocument();
    receipt.DefaultPageSettings.PaperSize = new PaperSize("Receipt", 350,
receiptLength);
    receipt.DocumentName = $"Order #{orderId} Receipt";
    receipt.PrintPage += new PrintPageEventHandler(this.PrintReceiptPage);
    printPreviewDialog1.Document = receipt;
    printPreviewDialog1.ShowDialog();
    receipt.Dispose();
}

private void PrintReceiptPage(object sender, PrintPageEventArgs e)
{
    Font myFont1 = new Font("Cooper", 18, FontStyle.Bold);
    Font myFont2 = new Font("Times New Roman", 10, FontStyle.Bold);
    Font myFont3 = new Font("Times New Roman", 8);
    int x = e.MarginBounds.X - 50;
    int y = e.MarginBounds.Y - 70;
    e.Graphics.DrawString("REINFORCEMENTS", myFont1, Brushes.Black, x, y);
    y += 35;
    e.Graphics.DrawString("120 St Georges Ave, Poole, BH12 4ND\n
0-7927-369-011", myFont3, Brushes.Black, x, y);
    y += 58;
    e.Graphics.DrawString($"ORDER: #{orderId}    DATE: {dateOfOrder}    CARD: xxxx
{GetRandom(1000, 9999)}", myFont3, Brushes.Black, x + 5, y);
    y += 18;
    e.Graphics.DrawString("===== ",
myFont3, Brushes.Black, -2, y);
    y += 30;
    e.Graphics.DrawString("NAME", myFont2, Brushes.Black, x, y);
    x = e.MarginBounds.X + 98;
    e.Graphics.DrawString("QTY", myFont2, Brushes.Black, x, y);
    x = e.MarginBounds.X + 158;
    e.Graphics.DrawString("PRICE", myFont2, Brushes.Black, x, y);
    y += 20;
    x = e.MarginBounds.X - 50;
    foreach (var product in productsList)
    {
        e.Graphics.DrawString(product, myFont3, Brushes.Black, x, y);
        y += 15;
    }
    x = e.MarginBounds.X + 98;
    y = 191;
}

```



```

        foreach (var qty in qtyList)
        {
            e.Graphics.DrawString(qty, myFont3, Brushes.Black, x, y);
            y += 15;
        }
        x = e.MarginBounds.X + 158;
        y = 191;
        priceList.Add(null);
        foreach (var price in priceList)
        {
            e.Graphics.DrawString(price, myFont3, Brushes.Black, x, y);
            if (price != null)
            {
                y += 15;
            }
        }
        y += 20;
        e.Graphics.DrawString("===== ",
myFont3, Brushes.Black, -2, y);
        y += 30;
        x = e.MarginBounds.X - 50;
        e.Graphics.DrawString($"SUBTOTAL", myFont3, Brushes.Black, x, y);
        y += 15;
        e.Graphics.DrawString($"VAT", myFont3, Brushes.Black, x, y);
        y += 15;
        e.Graphics.DrawString($"TOTAL", myFont2, Brushes.Black, x, y);
        y -= 30;
        x = e.MarginBounds.X + 158;
        e.Graphics.DrawString($"£{Math.Round(subtotal, 2)}", myFont3, Brushes.Black, x, y);
        y += 15;
        e.Graphics.DrawString($"£{Math.Round(VAT, 2)}", myFont3, Brushes.Black, x, y);
        y += 15;
        e.Graphics.DrawString($"£{Math.Round(total, 2)}", myFont2, Brushes.Black, x, y);
        y += 50;
        e.Graphics.DrawLine(new Pen(Color.Black, 3), new Point(40, y), new Point(40, y +
30));
        e.Graphics.DrawLine(new Pen(Color.Black, 3), new Point(310, y), new Point(310, y +
30));
        for (int i = 0; i < 125; i++)
        {
            x = GetRandom(43, 308);
            int width = GetRandom(1, 2);
            e.Graphics.DrawLine(new Pen(Color.Black, width), new Point(x, y), new Point(x, y
+ 30));
        }
        e.Graphics.DrawString("THANK YOU!", myFont1, Brushes.Black, e.MarginBounds.X - 7, y
+ 60);
    }

    private int GetRandom(int min, int max)
    {
        return Convert.ToInt32(rnd.Next(min, max));
    }
}

```

5.9 Class Sign In

frmSignIn

Reinforcements - Sign In

Sign In

Email address

Password

Remember me ☐ Sign in

First name

Surname

Email

Phone number

Company name

Password

Confirm password

Register

No account? [Create one!](#)
 Forgot your password? Call us!
 Contact number: +44 7927 369 011

C# code

```
namespace Coursework
{
    public partial class frmSignIn : Form
    {
        public frmSignIn()
        {
            InitializeComponent();
        }

        private void frmSignIn_Load(object sender, EventArgs e)
        {
            File.Delete("user details.txt");
            File.Delete("temp.txt");
            RegVisability(false);
        }

        bool successfullSignIn = false;
        bool regVisible = false;

        private string GetHashSHA256(string plainText)
        {
            string hashText = "";
            Encoding enc = Encoding.UTF8;
            SHA256Managed hash = new SHA256Managed();
            byte[] result = hash.ComputeHash(enc.GetBytes(plainText));
            foreach (Byte item in result)
            {
                hashText = hashText + item.ToString("X");
            }
            return hashText;
        }

        private void RegVisability(bool v)
        {
            txtFirstName.Visible = v;
            lblFirstName.Visible = v;
            txtSurname.Visible = v;
            lblSurname.Visible = v;
            txtEmailReg.Visible = v;
        }
    }
}
```

```

        lblEmailReg.Visible = v;
        txtPhoneNumber.Visible = v;
        lblPhoneNumber.Visible = v;
        txtCompanyName.Visible = v;
        lblCompanyName.Visible = v;
        txtPasswordReg1.Visible = v;
        lblPasswordReg1.Visible = v;
        txtPasswordReg2.Visible = v;
        lblPasswordReg2.Visible = v;
        btnRegister.Visible = v;
        picViewPassword2.Visible = v;
    }

    private void btnSignIn_Click(object sender, EventArgs e)
    {
        if (CheckValid() == true)
        {
            successfullSignIn = true;
            if (chkRemember.Checked == true)
            {
                RememberMe();
            }
            CreateTempFile();
            this.Close();
        }
    }

    private void CreateTempFile()
    {
        StreamWriter currentFile = new StreamWriter("temp.txt");
        File.SetAttributes("temp.txt", FileAttributes.Hidden);
        currentFile.WriteLine(txtEmail.Text);
        currentFile.WriteLine(isManager().ToString());
        currentFile.Close();
    }

    private bool isManager()
    {
        clsDBConnector dbConnector = new clsDBConnector();
        OleDbDataReader dr;
        string sqlStr;
        string boolDB = "";
        dbConnector.Connect();
        sqlStr = "SELECT Manager" +
            " FROM tblUser" +
            $" WHERE Email = '{txtEmail.Text}'";
        dr = dbConnector.DoSQL(sqlStr);
        while (dr.Read())
        {
            boolDB = dr[0].ToString();
        }
        dbConnector.Close();
        if (boolDB == "True")
        {
            return true;
        }
        else
        {
            return false;
        }
    }

    private void RememberMe()
    {
        StreamWriter currentFile = new StreamWriter("user details.txt");
        File.SetAttributes("user details.txt", FileAttributes.Hidden);
        currentFile.WriteLine(txtEmail.Text);
        currentFile.WriteLine(GetHashSHA256(txtPassword.Text));
        currentFile.Close();
    }

```

```

private bool CheckValid()
{
    if (txtPassword.Text == "" || txtEmail.Text == "")
    {
        MessageBox.Show("You have not entered anything for one or many fields.",
"Error");
        return false;
    }
    if (ExistingEmail(txtEmail.Text) == false)
    {
        MessageBox.Show("We couldn't find an account with that email. Try creating an
account.", "Error");
        return false;
    }
    if (GetCorrectPassword() == "")
    {
        MessageBox.Show("It seems like your account has been created by a manager.
Please give us a call to set up a password.", "Error");
        return false;
    }
    if (GetHashSHA256(txtPassword.Text) != GetCorrectPassword())
    {
        MessageBox.Show("Incorrect password.", "Error");
        return false;
    }
    return true;
}

private string GetCorrectPassword()
{
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    string correctPassword = "";
    dbConnector.Connect();
    sqlStr = "SELECT [Password]" +
            " FROM tblUser" +
            $" WHERE Email = '{txtEmail.Text}'";
    dr = dbConnector.DoSQL(sqlStr);
    while (dr.Read())
    {
        correctPassword = dr[0].ToString();
    }
    dbConnector.Close();
    return correctPassword;
}

private void lblRegister_Click(object sender, EventArgs e)
{
    if (regVisible == false)
    {
        RegVisability(true);
    }
    regVisible = true;
}

private void txtEmail_TextChanged(object sender, EventArgs e)
{
    if (txtEmail.Text == "")
    {
        lblEmail.Visible = true;
    }
    else
    {
        lblEmail.Visible = false;
    }
}

private void txtPassword_TextChanged(object sender, EventArgs e)
{
    if (txtPassword.Text == "")
    {
        lblPassword.Visible = true;
    }
}

```

```
        else
        {
            lblPassword.Visible = false;
        }
    }

private void txtFirstName_TextChanged(object sender, EventArgs e)
{
    if (txtFirstName.Text == "")
    {
        lblFirstName.Visible = true;
    }
    else
    {
        lblFirstName.Visible = false;
    }
}

private void txtSurname_TextChanged(object sender, EventArgs e)
{
    if (txtSurname.Text == "")
    {
        lblSurname.Visible = true;
    }
    else
    {
        lblSurname.Visible = false;
    }
}

private void txtEmailReg_TextChanged(object sender, EventArgs e)
{
    if (txtEmailReg.Text == "")
    {
        lblEmailReg.Visible = true;
    }
    else
    {
        lblEmailReg.Visible = false;
    }
}

private void txtPhoneNumber_TextChanged(object sender, EventArgs e)
{
    if (txtPhoneNumber.Text == "")
    {
        lblPhoneNumber.Visible = true;
    }
    else
    {
        lblPhoneNumber.Visible = false;
    }
}

private void txtCompanyName_TextChanged(object sender, EventArgs e)
{
    if (txtCompanyName.Text == "")
    {
        lblCompanyName.Visible = true;
    }
    else
    {
        lblCompanyName.Visible = false;
    }
}

private void txtPasswordReg1_TextChanged(object sender, EventArgs e)
{
    if (txtPasswordReg1.Text == "")
    {
        lblPasswordReg1.Visible = true;
    }
}
```

```

        else
        {
            lblPasswordReg1.Visible = false;
        }
    }

    private void txtPasswordReg2_TextChanged(object sender, EventArgs e)
    {
        if (txtPasswordReg2.Text == "")
        {
            lblPasswordReg2.Visible = true;
        }
        else
        {
            lblPasswordReg2.Visible = false;
        }
    }

    private void picViewPassword_MouseUp(object sender, MouseEventArgs e)
    {
        txtPassword.PasswordChar = '*';
        picViewPassword.Image = Image.FromFile("view_password.png");
    }

    private void picViewPassword_MouseDown(object sender, MouseEventArgs e)
    {
        txtPassword.PasswordChar = Convert.ToChar(0);
        picViewPassword.Image = Image.FromFile("hide_password.png");
    }

    private void picViewPassword2_MouseUp(object sender, MouseEventArgs e)
    {
        txtPasswordReg1.PasswordChar = '*';
        picViewPassword2.Image = Image.FromFile("view_password.png");
    }

    private void picViewPassword2_MouseDown(object sender, MouseEventArgs e)
    {
        txtPasswordReg1.PasswordChar = Convert.ToChar(0);
        picViewPassword2.Image = Image.FromFile("hide_password.png");
    }

    private void btnRegister_Click(object sender, EventArgs e)
    {
        if (CheckValidReg() == true)
        {
            clsDBConnector dbConnector = new clsDBConnector();
            string hashedPassword = GetHashSHA256(txtPasswordReg1.Text);
            string cmdStr = $"INSERT INTO tblUser (FirstName, Surname, Email, PhoneNumber,
CompanyName, [Password]) " +
                $"VALUES ('{txtFirstName.Text}', '{txtSurname.Text}',
'{txtEmailReg.Text}', '{txtPhoneNumber.Text}', '{txtCompanyName.Text}', '{hashedPassword}')";
            dbConnector.Connect();
            dbConnector.DoDML(cmdStr);
            dbConnector.Close();
            MessageBox.Show("Account created successfully. You can sign in now.",
"Registration completed");
        }
    }

    private bool CheckValidReg()
    {
        if (txtFirstName.Text == "" || txtSurname.Text == "" || txtEmailReg.Text == "" ||
txtPhoneNumber.Text == "" || txtPasswordReg1.Text == "" || txtPasswordReg2.Text == "")
        {
            MessageBox.Show("You have not entered anything for one or many fields.",
"Error");
            return false;
        }
        if (ExistingEmail(txtEmailReg.Text) == true)
        {
            MessageBox.Show("This email address is already used. Try to sign in.", "Error");
            return false;
        }
    }

```

```

string pattern = @"[A-Z][a-z]+";
Match tryToMatch = Regex.Match(txtFirstName.Text, pattern);
if (!tryToMatch.Success)
{
    MessageBox.Show("Invalid first name (e.g., Charlie).", "Error");
    return false;
}
tryToMatch = Regex.Match(txtSurname.Text, pattern);
if (!tryToMatch.Success)
{
    MessageBox.Show("Invalid surname (e.g., Black).", "Error");
    return false;
}
pattern = @"(?:[a-z0-9!#$%&'*/+=?^_`{|}~]+(?:\. [a-z0-9!#$%&'*/+=?^_`{|}~
]+)*|"(?:[a-z0-9!#$%&'*/+=?^_`{|}~\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x2f\x3b\x3d-\x7f]|\\[\x01-\x09\x0b\x0c\x0e-
\x7f])*")@(?:[a-z0-9](?:[a-z0-9]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9]*[a-z0-
9])?|\[(?:[a-z0-9]|0-4|0-9]|1[0-9]|0-9]|1[0-9]?[0-9])\.)\{3}(?:2(5[0-5]|0-4|0-9]|1[0-
9]|0-9]|1[0-9]?[0-9])|[a-z0-9]*[a-z0-9]:(?:[a-z0-9!#$%&'*/+=?^_`{|}~\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-
\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f])+)\]";
tryToMatch = Regex.Match(txtEmailReg.Text, pattern);
if (!tryToMatch.Success)
{
    MessageBox.Show("Invalid email address (e.g., charli3black999@gmail.com).",
"Error");
    return false;
}
pattern = @"^(?:[a-z0-9](?:[a-z0-9]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9]*[a-z0-
9])?|\[(?:[a-z0-9]|0-4|0-9]|1[0-9]|0-9]|1[0-9]?[0-9])\.)\{3}(?:2(5[0-5]|0-4|0-9]|1[0-
9]|0-9]|1[0-9]?[0-9])|[a-z0-9]*[a-z0-9]:(?:[a-z0-9!#$%&'*/+=?^_`{|}~\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-
\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f])+)\]";
tryToMatch = Regex.Match(txtPhoneNumber.Text, pattern);
if (!tryToMatch.Success)
{
    MessageBox.Show("Invalid phone number. (e.g., 07635483912)", "Error");
    return false;
}
if (txtPasswordReg1.Text != txtPasswordReg2.Text)
{
    MessageBox.Show("The passwords you have entered do not match.", "Error");
    return false;
}
return true;
}

private bool ExistingEmail(string email)
{
    clsDBConnector dbConnector = new clsDBConnector();
    OleDbDataReader dr;
    string sqlStr;
    int userExists = 0;
    dbConnector.Connect();
    sqlStr = $"SELECT UserID FROM tblUser WHERE Email = '{email}'";
    dr = dbConnector.DoSQL(sqlStr);
    while (dr.Read())
    {
        if (dr[0] != DBNull.Value)
        {
            userExists = 1;
        }
    }
    dbConnector.Close();
    if (userExists == 0)
    {
        return false;
    }
    else
    {
        return true;
    }
}

private void lblEmail_Click(object sender, EventArgs e)
{
    txtEmail.Focus();
}

```

```
private void lblPassword_Click(object sender, EventArgs e)
{
    txtPassword.Focus();
}

private void lblFirstName_Click(object sender, EventArgs e)
{
    txtFirstName.Focus();
}

private void lblSurname_Click(object sender, EventArgs e)
{
    txtSurname.Focus();
}

private void lblEmailReg_Click(object sender, EventArgs e)
{
    txtEmailReg.Focus();
}

private void lblPhoneNumber_Click(object sender, EventArgs e)
{
    txtPhoneNumber.Focus();
}

private void lblCompanyName_Click(object sender, EventArgs e)
{
    txtCompanyName.Focus();
}

private void lblPasswordReg1_Click(object sender, EventArgs e)
{
    txtPasswordReg1.Focus();
}

private void lblPasswordReg2_Click(object sender, EventArgs e)
{
    txtPasswordReg2.Focus();
}

private void frmSignIn_FormClosing(object sender, FormClosingEventArgs e)
{
    if (successfullSignIn == false)
    {
        Application.ExitThread();
    }
}
}
```


5.10 Class DBConnector

C# code

```
namespace Coursework
{
    class clsDBConnector
    {
        OleDbConnection conn = new OleDbConnection();
        string dbProvider;
        string dbSource;
        string workingDirectory = Environment.CurrentDirectory;

        public void Connect()
        {
            dbProvider = "Provider=Microsoft.ACE.OLEDB.12.0;";
            dbSource = @"Data Source = " + workingDirectory + @"\dbCW.accdb" ;
            conn.ConnectionString = dbProvider + dbSource;
            conn.Open();
        }

        public void Close()
        {
            conn.Close();
        }

        public void DoDML(string dmlString)
        {
            OleDbCommand cmd;
            cmd = new OleDbCommand(dmlString, conn);
            cmd.ExecuteNonQuery();
        }

        public OleDbDataReader DoSQL(string sqlString)
        {
            OleDbCommand cmd;
            cmd = new OleDbCommand(sqlString, conn);
            return cmd.ExecuteReader() ;
        }
    }
}
```

5.11 Class Program

C# code

```
namespace Coursework
{
    internal static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new MyAppContext());
        }

        public class MyAppContext : ApplicationContext
        {
            public MyAppContext()
            {
                bool rememberMe = File.Exists("user details.txt");
                if (rememberMe)
                {
                    frmMain mainForm = new frmMain();
                    mainForm.FormClosed += MainForm_FormClosed;
                    mainForm.Show();
                }
                else
                {
                    frmSignIn frmSignIn = new frmSignIn();
                    frmSignIn.FormClosed += SignInForm_FormClosed;
                    frmSignIn.Show();
                }
            }

            private void SignInForm_FormClosed(object sender, FormClosedEventArgs e)
            {
                frmMain mainForm = new frmMain();
                mainForm.FormClosed += MainForm_FormClosed;
                mainForm.Show();
            }

            private void MainForm_FormClosed(object sender, FormClosedEventArgs e)
            {
                ExitApp();
            }
        }

        internal static void ExitApp()
        {
            Application.Exit();
        }
    }
}
```